SW Applications in Aerospace Engineering 1
Term project Report
부제: 항소개팅

5조: 김찬혁[1], 김채은[1], 유승표[1], 이용신[2], 홍재우[1]
물리천문학과 전공 (항공우주공학 복수전공)[1], 항공우주공학 전공[2]

1. 서론

1.1 배경

현대 사회는 정보 기술의 발전으로 데이터 수집 및 활용이 매우 편리해졌다. 그중에서도 GNSS (Global Navigation Satellite System)는 인공위성에서 방송된 신호를 수신하는 장치를 이용하여 사용자가 자신의 정확한 시각과 3-D 위치 및 속도를 실시간으로 제공하는 시스템이다. 위성 시스템 중 미국이 구축한 GPS (Global Positioning System)가 가장 흔히, 또 유용하게 활용되고 있다. GPS는 4개의 위성에서 보내는 신호를 수신하여 신호의 도달 시간 차이를 계산하여 거리를 계산하고, 거리를 이용해 삼각측량을 통해 3차원 위치 계산을 해주는 시스템이다. 이러한 GPS 데이터는 다양한 응용 분야에서 활용되고 있으며, 내비게이션, 위치 기반 서비스와 교통 관리 그리고 측량 등에서 흔히 활용되고 있다.

MATLAB은 데이터 분석, 시뮬레이션, 수치 계산 등에 유용한 프로그래밍 환경이다. MATLAB을 통해 시간, 경도, 위도, 고도 등의 정보를 포함하는 GPS 데이터를 불러온 후 코드 작성 및 함수를 통해 데이터를 분석하고, 파일로 저장하거나 다양한 방식으로 데이터를 처리할 수 있다. 또한, GUI (Graphical User Interface)를 활용하여 사용자가 데이터를 시각적으로 처리하거나 조작할 수 있도록 하는 인터페이스를 제공함으로써 사용자가 코드를 작성하지 않고도 간단한 조작으로 사용자와 상호 작용할 수 있도록 한다.

이와 같이 GPS 데이터와 MATLAB을 이용하여 데이터 수집, 분석과 처리를 통해 다양한 곳에서 유용하게 활용될 수 있다.

1.2 목적

'항소개팅'은 GPS 데이터 정보를 통해 사용자의 일과를 분석하여 시간표와 같은 일과에 맞는 사람을 소개해 주는 프로그램이다. 사용자의 일상에 맞추어 만나고자 하는 목적에 따라 다른 사용자를 이어주는 역할을 한다. 사용자가 쉽게 얻을 수 있는 GPS 데이터를 이용하여 사용자의 위치, 시간 데이터를 얻어 프로그램에 활용할 수 있다.

인공위성에서 송신되는 GPS 신호를 수신하여 사용자의 시간에 따른 위치 정보를 얻는다. 스마트폰으로 GPS 데이터를 받아 데이터를 바탕으로 사용자 하루 일과의 이동 경로

를 파악하고, 시간표와 일상을 파악하기 위해 MATLAB에서 효율적인 코드 작성을 한다.
사용자가 원하는 목적에 맞는 만남을 위해서는 데이터베이스와의 비교, 분석 과정을 거친다. 사용자의 원하는 만남의 목적과 만나고자 하는 사용자의 데이터들을 비교하여 일치, 불일치하는 사용자의 데이터를 분류하는 과정이 필수적이다.
GUI를 이용해 사용자가 직접 코드를 작성하지 않고 단순한 조작만으로 사용자와 코드가 상호작용할 수 있는 인터페이스를 생성한다. 사용자에게 친화적인 인터페이스를 만들기 위해 한눈에 들어오는 인터페이스를 제작하고, 인터페이스를 통해 간단한 조작으로 데이터를 주고받아 결과적으로 사용자가 원하는 매칭을 만들어 낼 수 있도록 한다.

2. 본론

2.1 배경 지식

그림 1 GNSS 3차원 가상 이미지 [1]

GNSS (Global Navigation Satellite System, 위성 항법 시스템)는 다수의 인공위성에서 방송된 신호를 수신하는 장치를 이용하여 사용자가 자신의 정확한 3차원 위치와 속도 그리고 시간을 계산할 수 있도록 하는 시스템이다. 위성항법 시스템은 지구궤도를 도는 항법 인공위성을 의미하는 우주 부분 (Space segment), 지상 안테나와 센터들로 이루어져 있으며 항법위성을 추적하고 감시하는 제어 부분 (Control segment), 위성항법신호를 수신하여 위치/속도/시간을 계산하는 사용자인 사용자 부분 (User segment)으로 구성되어 있다.
사용자의 위치, 속도, 시간을 계산하기 위해서 지구 궤도를 돌고 있는 위성에게서 지속적으로 위성의 위치와 송신된 시간 정보가 포함되어 있는 신호를 받는다. 3차원 위치 결정을 위해서는 4개의 항법위성이 필요하기에 수신기는 4개의 위성으로부터 신호를 각각 수신한다. 수신기는 항법위성으로부터 수신기에 도달하는데 소요되는 시간을 측정한다. 송신한 시간과 수신된 시간 차이 (소요 시간)와 여러 오차 요소들을 제거한 값으로

각 항법위성과 수신기 사이의 거리를 계산할 수 있다. 각 항법위성과의 거리를 통해 삼각측량법을 이용하여 결국 사용자의 3차원 위치를 특정할 수 있다. 수신기에서 정보를 외부로 송출하기 위한 형식 중 하나가 바로 NMEA 파일 형식이다.

그러나 위성항법 시스템에는 대기 오차, 다중경로, 주변 전자기 신호의 간섭에 의한 오차 등 여러 오차가 발생한다. 그중 다중경로 오차는 위성 신호가 건물과 같은 장애물에 반사되어서 발생한다. 따라서 얼마나 다양한 각도에서 위성 신호를 수신하는지에 따라서 위치 측정의 정밀도가 달라진다.

실내에서는 물리적 장애물과 전자제품의 신호 간섭으로 위성 신호가 약해지고 위치 계산에 어려워진다. 따라서 실내에서 GNSS 신호만으로 위치를 추적하기에는 어려움이 있다. 이러한 경우 기지국과의 통신을 통한 위치추적 기술인 SBAS (Satellite Based Augmentation System)항법과 RTK (Real Time Kinematic) GNSS 기술을 사용해 위성항법 위치추적 오차를 줄일 수 있다. 실내용 레이더나 블루투스 (Bluetooth), 와이파이 (Wi-Fi) 등을 사용하는 실내 위치 확인 시스템 (Indoor Positioning System)을 사용하는 방법을 선택할 수 있다. 따라서 실내에서 위성 신호 수신이 어렵게 되면 스마트폰의 IMU 센서를 통한 관성항법 기술을 사용해 위치를 추적한다. 그러나 관성항법 위치 추적에도 오차가 발생하고, 이는 시간이 지날수록 축적되므로 위치정확도가 점차 떨어지게 된다.

본 프로그램의 개발 초기 계획은 NMEA 파일 형식으로 데이터를 분석 및 처리하려고 하였으나, 사용한 tracking 어플 (Guru Maps)이 NMEA 파일 형식을 지원하지 않았고, CSV 파일 형식에서 NMEA 파일 형식으로 외부에서 변환 시 GPWPL으로 변환되며 해당 형식은 고도와 시간 정보가 포함되지 않는 문제가 있어 데이터 처리를 CSV 파일 형식을 바탕으로 진행했다. CSV 파일(Comma Separated Values)은 데이터를 표 형식으로 저장하는 텍스트 파일 형식이다. 숫자와 문자만 포함될 수 있고, 데이터를 표 형식으로 구성할 수 있다. 데이터 구분을 쉼표로 하기에 매우 간단하고 가독성 높은 파일 형식이다.

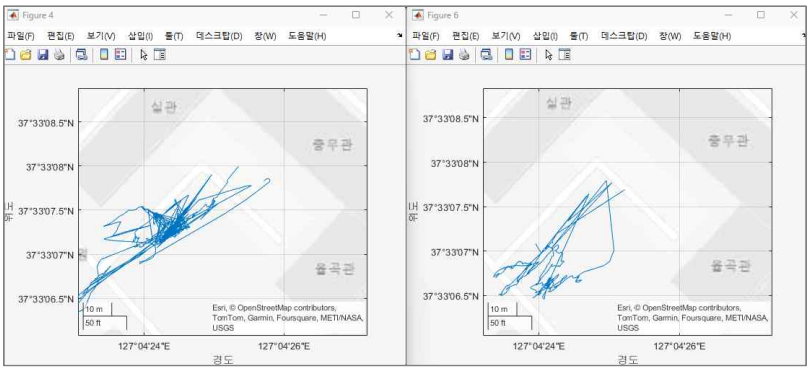


그림 2 충무관 GNSS 정보 트래킹 결과

위 사진은 충무관 1층 건물에서의 수업 시간동안 GNSS 정보를 트래킹한 모습이다. 충무관에서는 GNSS 기반 위치오차가 클 뿐만 아니라 위성 신호의 수신이 원활하지 못해 위치오차가 누적해서 증가하는 모습을 보인다. 이 프로젝트에서 GNSS 정보를 통해 건물별 시간표 데이터를 구성하기 위해 이러한 위치 데이터의 오차는 필수적으로 해결해야 할 과제이다.

2.2 프로그램 구성

본 프로그램은 9가지 주요 화면으로 구성된다. 시작 화면, 회원가입, 로그인, 시간표, 메인 화면, 정보 수정, 선호 설정, 매칭, 결과 창으로 구분된다. 각 화면의 기능과 흐름은 다음과 같다.

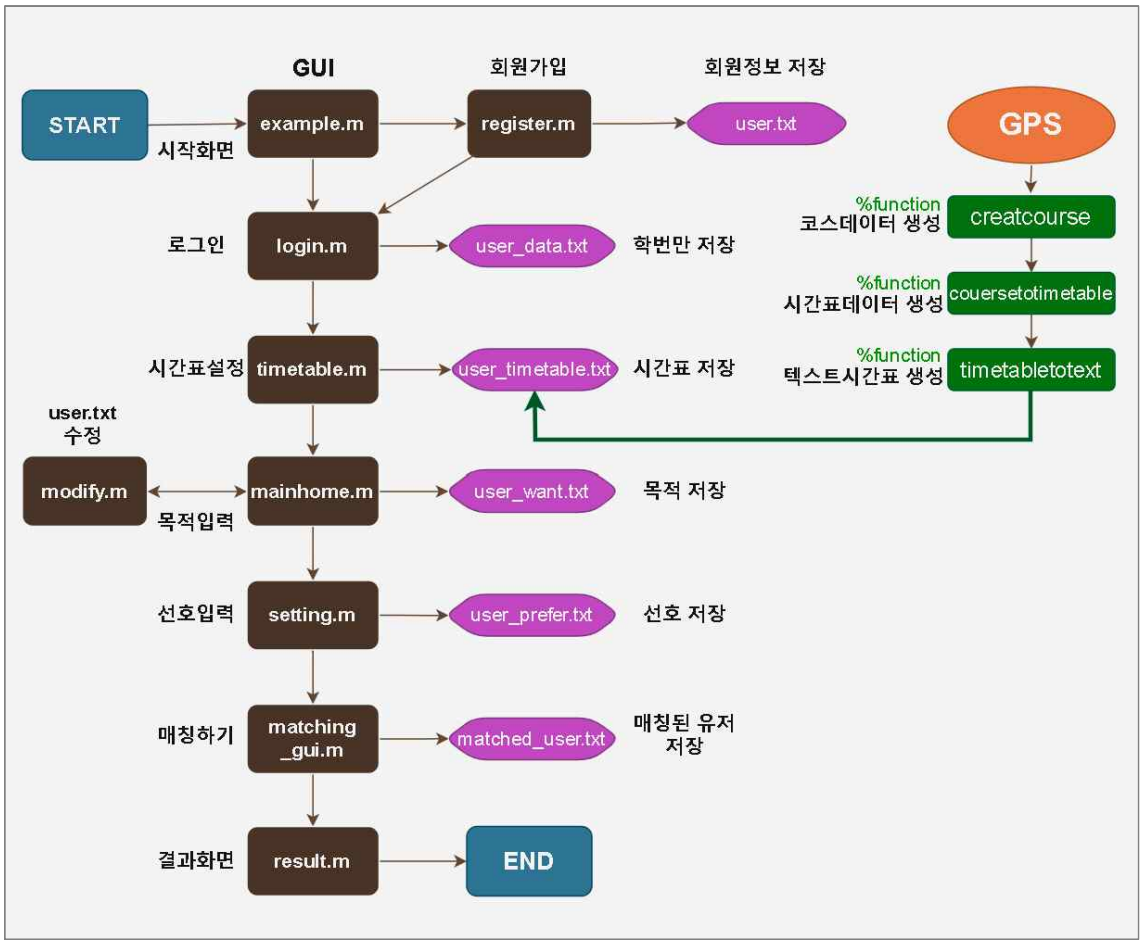


그림 3 항소개팅 flow-chart

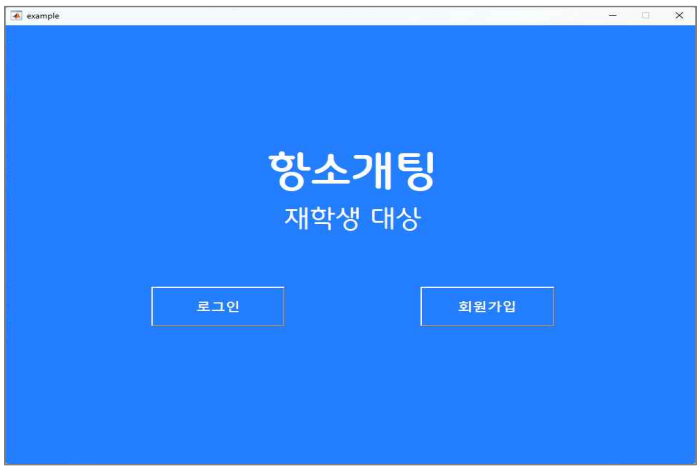


그림 4 시작 화면

프로그램 실행 후 가장 먼저 시작 화면이 표시된다. 여기서 사용자는 로그인 또는 회원가입 중 하나를 선택할 수 있다.

그림 5 회원가입

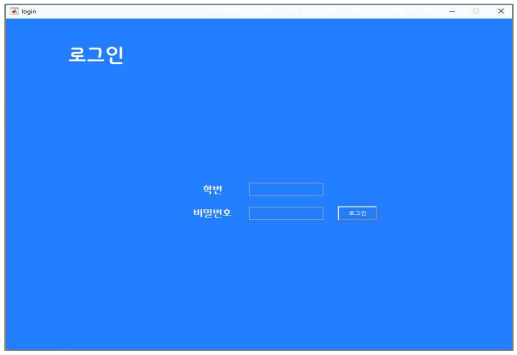


그림 6 로그인

회원가입 창에서는 사용자가 학번, 비밀번호, 전공, 성별, 학년, 카카오톡 ID를 입력한다. 전공, 성별, 학년은 팝업 창을 통해 선택하며, 학번과 비밀번호는 직접 입력한다. 입력 완료 후, '회원 정보가 저장되었습니다!'라는 문구와 함께 로그인 화면으로 이동한다. 만약 이미 등록된 학번일 경우, '이미 등록되어 있는 아이디입니다.'라는 메시지가 출력된다.

로그인 화면에서 사용자는 학번과 비밀번호를 입력한다. 입력된 정보가 데이터베이스와 일치하면 '로그인 성공' 메시지가 출력되며 다음 화면으로 이동한다. 학번이나 비밀번호가 틀릴 경우, '학번 또는 비밀번호가 잘못되었습니다.'라는 경고 문구가 표시된다.

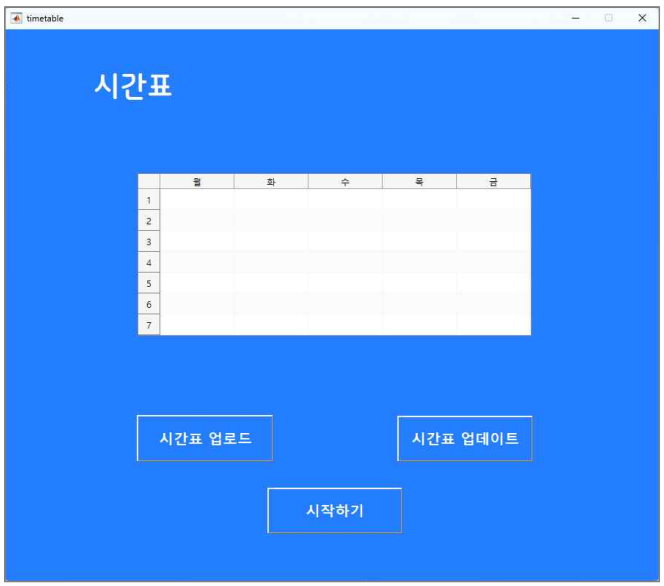


그림 7 시간표 설정

시간표 화면은 다음 세 가지 기능으로 구성된다.

- 시간표 업로드: 파일 업로드 링크 (사전에 설정된 구글 드라이브)에 연결, 사용자가 시간표 데이터를 업로드할 수 있도록 함 (파일 형식은 '학번_요일.csv'),
- 시간표 업데이트: 기존 시간표 데이터를 읽어 화면에 표시,
- 시작하기: 시간표 갱신 후 메인 화면으로 이동.

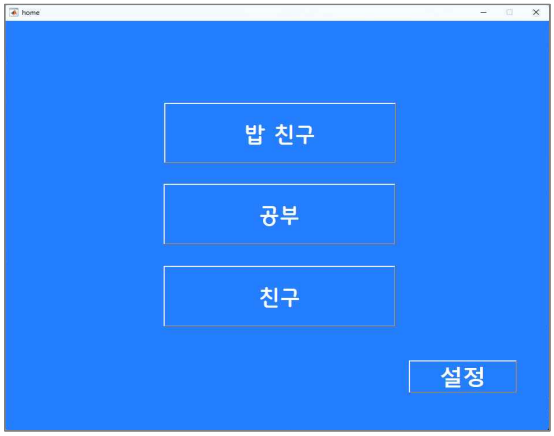


그림 8 목적 입력 (메인 화면)

그림 9 정보 수정

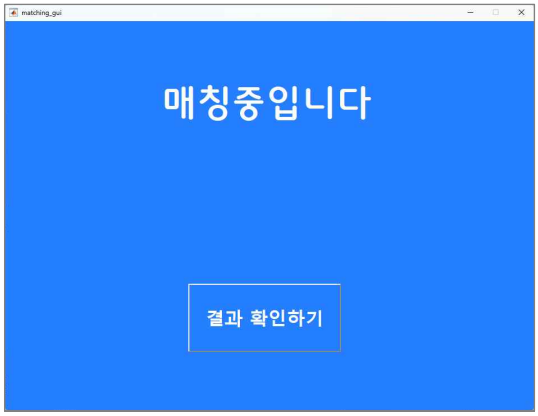
메인 화면에서는 사용자의 목적에 따라 밥 친구, 공부, 친구를 선택하거나 설정을 변경할 수 있다. 각 항목을 선택하면 자동으로 선호 만남 정보 (user_want.txt)의 업데이트가 이루어지며, 다음 화면으로 진행된다. 설정 버튼을 선택하면 정보 수정 화면으로 이동한다.

정보 수정 화면에서는 회원가입 시 입력했던 정보를 변경할 수 있다. 변경 완료 후 '수정 완료' 버튼을 누르면 다시 메인 화면으로 돌아간다.

그림 10 선호 입력

선호 설정 화면에서는 다음 항목을 선택하여 매칭에 필요한 기준을 설정할 수 있다.

- 성별: 같은 성별 (남자/여자) 또는 전체 (all) 중 선택,
- 학과: 같은 학과 (same), 다른 학과 (other), 또는 전체 (all) 중 선택.
- 학년: 1학년부터 5학년까지 개별 선택 가능, 또는 전체 (all) 선택,
- 만나고 싶은 요일: 월요일부터 금요일 중 원하는 요일 선택.



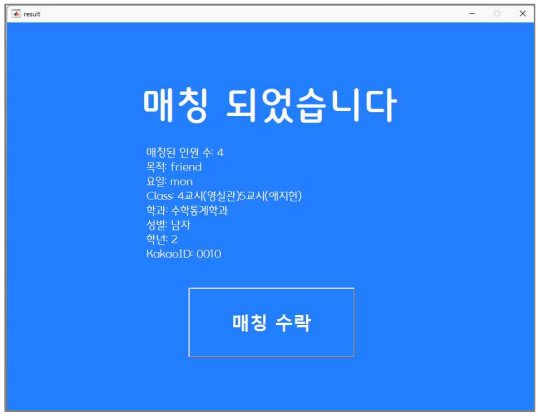


그림 11 매칭 로딩 화면

그림 12 매칭 결과 화면

매칭에서는 로그인한 사용자의 선호 정보를 기반으로, 데이터베이스에 있는 다른 사용자들과의 매칭을 진행한다. 선호 정보와 시간표를 비교하여 조건에 맞는 사람을 찾아내며, 매칭된 사용자 정보를 저장한다.

결과 창에서는 매칭된 상대방의 정보와 카카오톡 ID가 표시된다. 이를 통해 사용자는 상대방에게 직접 연락할 수 있다. 매칭 결과가 없을 경우 '조건에 맞는 사람을 찾을 수 없습니다.' 라는 메시지가 표시된다.

각 프로그램의 실행 흐름은 다음과 같이 나누어 볼 수 있다:

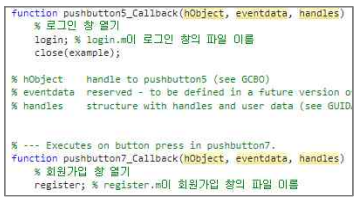
- 시작 화면 → 로그인 또는 회원가입 선택,
- 회원가입 → 회원정보 저장 → 로그인 화면으로 이동,
- 로그인 성공 → 시간표 화면으로 이동,
- 시간표 설정 → 메인 화면으로 이동,
- 메인 화면 → 선호 설정 또는 매칭 진행,
- 선호 설정 완료 → 매칭 진행,
- 매칭 결과 → 결과 창에 매칭 정보 표시.

이와 같은 구성으로 프로그램은 사용자의 시간표와 선호 정보를 활용해 효율적인 매칭 경험을 제공한다.

2.3 세부 컴포넌트 기능

2.3.1 프로그램 시작

로그인과 회원가입은 각각의 버튼을 눌렀을 때 콜백 함수를 통해 해당 GUI가 나타나도록 구현되었다.



```
function pushbutton5_Callback(hObject, eventdata, handles)
    % 로그인 창 열기
    login; % login.m이 로그인 창의 파일 이름
    close(example);

% hObject    handle to pushbutton5 (see GCBO)
% eventdata  reserved - to be defined in a future version o
% handles    structure with handles and user data (see GUID

% --- Executes on button press in pushbutton7.
function pushbutton7_Callback(hObject, eventdata, handles)
    % 회원가입 창 열기
    register; % register.m이 회원가입 창의 파일 이름
```

그림 13 프로그램 시작 GUI 내부 코드

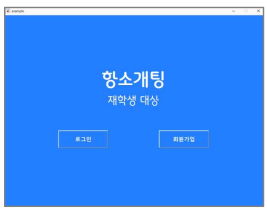


그림 14 exampe fig. GUI 창

2.3.2 회원가입

회원가입은 register.m을 통해서 이루어진다. 해당 GUI에서 사용가 학번, 비밀번호, 전공, 성별, 학년, 카카오톡 ID를 모두 입력하면 그 정보가 저장될 수 있도록 한다. 이 단계는 크게 4가지의 단계로 나누어진다. 입력된 사용자의 입력 정보를 저장한다, 필요할 경우 입력 정보 및 전체 회원의 정보를 구조화하고, 전체 회원의 학번과 입력된 사용자의 학번을 비교하여, 중복된 경우가 없을 때 회원가입을 하려는 사용자의 정보를 저장하는 방식으로 이루어진다.

그림 15 register fig.
GUI 창

첫 번째로, GUI에 입력한 정보를 저장한다. 정보를 전부 입력하지 않았을 경우, '모든 정보를 입력해야 합니다!' 라는 알림 창과 함께 다시 입력할 수 있도록 한다. 모든 정보가 입력된 경우, 전체 회원 정보 데이터베이스인 user.txt 파일이 존재하는지 확인한다. user.txt 파일이 존재하지 않을 경우 파일을 생성 후, 입력된 정보를 저장한다. 회원가입에 필요한 전체 정보를 바르게 입력했다면, 개인정보 및 위치정보 제공에 대한 동의 확인 안내메시지 창과 함께 동의 여부를 구한다. 이때 동의하지 않는 경우 회원가입이 무산되며 프로그램이 종료된다. 동의한 경우, '회원 정보가 저장되었습니다!' 라는 알림 창을 띄우고, 창을 지울 때, 로그인을 위한 GUI가 실행된다.

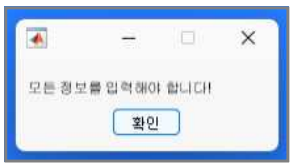


그림 16 정보 부족 안내 창

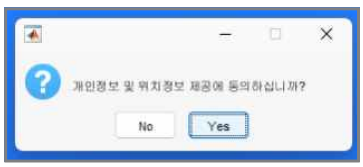


그림 17 개인정보 동의 창

그림 18 회원가입 완료 창

```
function pushbutton6_Callback(hObject, eventdata, handles)
    % 1. 입력값 가져오기
    stid = get(handles.edit_stid, 'String');
    pw = get(handles.edit_pw, 'String');
    major = get(handles.popupmenu_major, 'String');
    major = major{get(handles.popupmenu_major, 'Value')}; % 선택된 값
    gender = get(handles.popupmenu_gender, 'String');
    gender = gender{get(handles.popupmenu_gender, 'Value')};
    grade = get(handles.popupmenu_grade, 'String');
    grade = grade{get(handles.popupmenu_grade, 'Value')};
    kakao = get(handles.edit_kkid, 'String');
    if isempty(stid) || isempty(kakao) || isempty(pw)
        msgbox('모든 정보를 입력해야 합니다!');
    else
```

그림 19 입력값 저장

```
% 2. 데이터 파일에 저장
filename = 'user.txt'; % 저장할 파일 이름
if ~isfile(filename) %파일이 존재하지 않을경우 파일 생성
    fid = fopen(filename, 'wt');
    fprintf(fid, '%s\t%s\t%s\t%s\t%s\t%s\n', ...
        stid, pw, major, gender, grade, kakao);
    fclose(fid);

    str=questdlg('개인정보 및 위치정보 제공에 동의하십니까?','','No','Yes','Yes');
    if strcmp(str,'Yes')
       fprintf(fid,'\n%s\t%s\t%s\t%s\t%s\t%s\n',stid,pw,major,gender,grade,kakao);

       h = msgbox('회원 정보가 저장되었습니다!', '완료');
       uiwait(h);
       run login.m;
       close(register);
       close(example);
    else
        close(register);
    end
```

그림 20 회원 정보 저장

```
user.txt
21010480	1234	물리천문학과 남자	2	qwer
```

그림 21 회원가입 후 정보 저장 결과 (아무 회원도 없는 경우)

파일이 이미 존재할 경우, user.txt 파일을 읽기 모드로 열고 구조화한다. user 구조에는 회원가입을 시도하는 사용자의 정보를 제외한 회원들의 정보 (학번, 비밀번호, 전공, 성별, 학년, 카카오톡 ID)가 stid, pw, major, gender, grade, kakao의 필드로 저장된다. 이후, 파일에 이어서 작성하기 위해 append 모드로 연다. stid 필드에 저장된 모든 회원들의 학번과, 입력한 사용자의 학번을 비교한다. 학번이 겹치지 않을 경우, 입력된 정보를 user.txt 파일에 추가한다. '회원 정보가 저장되었습니다!' 라는 알림 창을 띄우고, 창을 지울 때, 로그인을 위해 login.m을 실행한다. 입력한 학번의 user 정보가 있는 경우 이미 가입되었다는 안내메시지를 출력한다.

```
% Users 정보 파일 read 모드로 열기
fid = fopen(filename, 'rt');

% Users의 정보를 담을 빈 struct 구조 생성
user = {};
i = 1; % index 용 숫자

while ~feof(fid)   % 파일의 끝에 도달하지 않았으면 반복
    line = fgetl(fid);   % 해당 열 문자를 받기

    if line == -1   % 마지막 열일 경우 반복문 종료
        break
    end

    linedata = strsplit(line,'\t');   % User의 정보를 tap 을 기준으로 정보를 쪼개고 user struct에 저장

    user(i).stid = linedata{1};
    user(i).pw = linedata{2};
    user(i).major = linedata{3};
    user(i).gender = linedata{4};
    user(i).grade = linedata{5};
    user(i).kakao = linedata{6};

    i = i+1;  % 다음 user struct에 저장하기 위해 index 업데이트
end
```

그림 22 회원들의 정보 구조화

```
% user.txt에 이어서 작성하기 위해 append 모드로 열기
fid = fopen(filename, 'at');

if numel(user) == 0   % user struct가 빈 struct면 저장된 정보가 아무것도 없으므로 정보 추가

    str=questdlg('개인정보 및 위치정보 제공에 동의하십니까?','','No','Yes','Yes');

    if strcmp(str,'Yes')
        fprintf(fid,'\n%s\t%s\t%s\t%s\t%s\t%s\n',stid,pw,major,gender,grade,kakao);

        h = msgbox('회원 정보가 저장되었습니다!', '완료');
        uiwait(h);
        run login.m;
        close(register);
        close(example);
    else
        close(register);
        close(example);
    end


else
    index = find(strcmp({user.stid},stid));  % 기존 데이터에 입력한 학번의 user가 존재하는지 확인
    if numel(index) == 0   % 입력한 학번의 user 정보가  없는 경우 추가
        str=questdlg('개인정보 및 위치정보 제공에 동의하십니까?','','No','Yes','Yes');

        if strcmp(str,'Yes')
            fprintf(fid,'%s\t%s\t%s\t%s\t%s\t%s\n',stid,pw,major,gender,grade,kakao);
            h = msgbox('회원 정보가 저장되었습니다!', '완료');
            uiwait(h);
            run login.m;
            close(register);
            close(example);
        else
            close(register);
            close(example);
        end


    else   % 입력한 학번의 user 정보가 있는 경우 이미 가입되었다는 안내메세지 출력
        msgbox('이미 등록되어 있는 아이디입니다.');

    end
end
```

그림 23 파일에 입력한 정보 저장

```
user.txt
21010512	1111	물리천문학과	여자	4	0000
23010018	0000	물리천문학과	남자	2	0001
23010029	0000	수학통계학과	남자	2	0002
23020018	0000	물리천문학과	여자	2	0003
23020029	0000	수학통계학과	여자	2	0004
23120018	0000	물리천문학과	여자	2	0005
23120029	0000	수학통계학과	여자	2	0006
21010514	0000	물리천문학과	남자	2	0007
21110514	0000	수학통계학과	여자	2	0008
23010118	0000	물리천문학과	남자	2	0009
23010129	0000	수학통계학과	남자	2	0010
23020118	0000	물리천문학과	여자	2	0011
23020129	0000	수학통계학과	여자	2	0012
23114413	1111	우주항공드론공학부 항공시스템공학전공	남자	1	1111
210101512	2222	물리천문학과	여자	4	1111
2101051442	1111	물리천문학과	여자	4	1111
```

그림 24 다양한 사용자들의 회원가입 후 전체 회원 정보 저장 예시

2.3.3 로그인

로그인 GUI는 저장된 학번과 비밀번호 입력을 통해 사용자가 항소개팅에 접속할 수 있게 해주는 유저 인터페이스이다. 사용자가 GUI 상에 학번과 비밀번호를 입력하면 get(handles.edit, ‘String’) 함수를 사용해 이를 stid 와 pw 변수에 저장한다. 이를 모든 회원 정보가 들어있는 user.txt 파일에서 탐색하고, user.txt 파일에 사용자가 입력한 정보가 들어있을 때 이를 로그인 한 사용자의 정보를 저장하는 user_data.txt 파일에 저장한다. 이후, time_table GUI를 실행한다. 잘못된 정보를 저장한 경우, ‘학번 또는 비밀번호가 잘못되었습니다.’ 안내메시지 창이, 올바른 정보를 입력한 경우 ‘로그인 성공’ 안내메시지 창이 뜬다. 아래의 사진은 로그인에 성공한 경우와 실패한 경우를 조건문으로 구분한 코드이다.

```
if login_success


    % user_data.txt에 새로운 학번 저장

    userdata_filename = 'user_data.txt';
    % 기존 내용을 지우고 새로운 학번을 저장
    fid = fopen(userdata_filename, 'wt'); % 쓰기 모드로 열기 (기존 내용 삭제)
    if fid == -1
        errordlg('user_data.txt 파일을 열 수 없습니다. 파일 경로를 확인하세요.', '파일 오류');
        return;
    end
    fprintf(fid, '%s\t', stid); % 새로운 학번 추가
    fprintf(fid, '%s\t', pw);
    fprintf(fid, '%s\t', user(j).major);
    fprintf(fid, '%s\t', user(j).gender);
    fprintf(fid, '%s\t', user(j).grade);
    fprintf(fid, '%s', user(j).kakao);


    fclose(fid);
    disp('user_data.txt 파일이 업데이트되었습니다.');
    str=questdlg('개인정보 및 위치정보 제공에 동의하시나요','','No','Yes','Yes');
    if strcmp(str,'Yes')


    close(login);
    close(example);
    time_table;

    h=msgbox('로그인 성공!', '성공');
    else
        close(login);
    close(example);
    end
else
    msgbox('학번 또는 비밀번호가 잘못되었습니다.', '오류', 'error');
```

그림 25 새로운 학번 저장 1

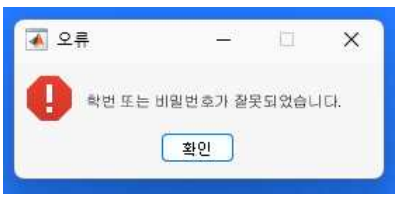


그림 28 정보 기입 오류 안내

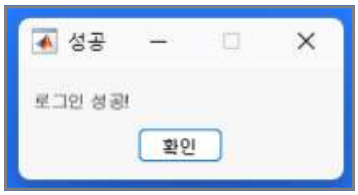


그림 29 로그인 성공 안내

2.3.4 회원 정보 수정

회원가입은 modify.m을 통해서 이루어진다. 해당 GUI에서 사용자는 비밀번호, 전공, 학년, 카카오톡 ID를 모두 입력하면 그 정보가 기존의 저장된 정보에서 수정될 수 있도록 한다. 이 단계는 크게 4가지의 단계로 나누어진다. 사용자의 입력 정보를 저장한다. 전체 회원들의 정보를 구조화하고, 사용자의 학번과 전체 회원의 학번을 비교하여, 로그인 사용자의 기존 정보를 지우고 수정된 정보로 새로 저장한다.

그림 30 modify fig, ‘수정 완료’를 누르면 해당 아래의 코드가 돌아감

첫 번째로, GUI 상에 입력한 사용자의 수정 정보를 저장한다. 정보를 전부 입력하지 않았을 경우, ‘모든 정보를 입력해야 합니다!’라는 알림 창과 함께 다시 입력할 수 있도록 한다. 모든 정보가 입력된 경우, user.txt 파일을 읽기 모드로 열고 구조화한다. usernew 구조에는 모든 유저들의 정보(학번, 비밀번호, 전공, 성별, 학년, 카카오톡 ID)가 stid, pw, major, gender, grade, kakao의 필드로 저장된다. 이후 user_data.txt 파일을 읽어 로그인 학번의 정보를 가져온다. user_data.txt 파일은 로그인 할 경우 반드시 로그인 학번으로 저장되기에, 파일이 없는 경우엔 접속 오류로 생각한다.

```
% 1. 입력값 가져오기
pw = get(handles.edit_pw, 'String');
major = get(handles.popupmenu_major, 'String');
major = major{get(handles.popupmenu_major, 'Value')}; % 선택된 값
gender = get(handles.popupmenu_gender, 'String');
gender = gender{get(handles.popupmenu_gender, 'Value')};
grade = get(handles.popupmenu_grade, 'String');
grade = grade{get(handles.popupmenu_grade, 'Value')};
kakao = get(handles.edit_kkid, 'String');
if isempty(kakao) || isempty(pw)
    msgbox('모든 정보를 입력해야 합니다!');
```

그림 31 입력값 저장

```
% 2. 데이터 파일에 저장
% User data base path
filename = 'user.txt';
fid = fopen(filename,'rt');
i = 1;
usernew = {}; % 기존 데이터 담을 빈 struct

while ~feof(fid)
    line = fgetl(fid);

    if line == -1
        break
    end

    linedata = strsplit(line,'\t');   % User의 정보를 tap 을 기준으로 정보를 쪼개고 user struct에 저장
    usernew(i).stid = linedata{1};
    usernew(i).pw = linedata{2};
    usernew(i).major = linedata{3};
    usernew(i).gender = linedata{4};
    usernew(i).grade = linedata{5};
    usernew(i).kakao = linedata{6};

    i = i +1;
end
fclose(fid);
```

그림 32 정보 구조화

```
    %   정보 DB에 새로 입력
    fid = fopen(filename,'wt');
    % 새 유저 DB 파일 작성
    for k = 1:length(usernew)
        fprintf(fid,'%s\t%s\t%s\t%s\t%s\t%s\n',usernew(k).stid,usernew(k).pw,...
            usernew(k).major,usernew(k).gender, usernew(k).grade, ...
            usernew(k).kakao);
        disp('회원 정보가 수정되었습니다.')
    end

    % 수정된 유저 정보 추가 작성
    fprintf(fid,'%s\t%s\t%s\t%s\t%s\t%s\n',stid,pw,major,gender,grade,kakao);
    fclose(fid);
    run mainhome.m;
    close(modify);
end
```

그림 33 수정된 정보를 추가한 파일 재생성

usernew 구조의 stid 필드에서 학번이 로그인 학번과 같은 경우, 해당하는 인덱스를 저장한다 (usernew 구조에서 사용자의 기존 정보가 저장되어 있는 순서 저장). 인덱스를 이용해 usernew 구조에서 사용자의 기존 정보를 모두 지운다, 파일을 쓰기 모드로 열어 usernew에서 추출한 사용자를 제외한 모든 회원의 정보에 수정된 정보를 추가하여 user.txt 파일을 재생성한다. 이후 선호 선택 (1) 창으로 돌아간다.

```
% 학번이 같은 로그인한 학번과 같은 경우 해당하는 index 저장 (1건만 존재)
index = find(strcmp({usernew.stid},stid));;
usernew(index) = []; % 중복되는 index 삭제
```

그림 34 로그인 유저의 기존정보 제거

2.3.5 시간표

2.3.5.a 오차 수정을 위한 건물 경계값 설정

건물명을 반환할 때 보정된 위치 데이터값이 건물 경곗값 내에 포함되는 빈도수를 측정하는 방법을 사용했다. 건물 경계의 위도, 경도 값은 google 지도 홈페이지를 통해 구하였다. 각 건물의 경계 위치는 아래 그림에서 지도 위에 나타내었다. Google 지도와 MATLAB이 보여주는 지도 간에 차이가 있어, MATLAB 그림지도 상의 건물과 위치 점이 다르게 나타난다.

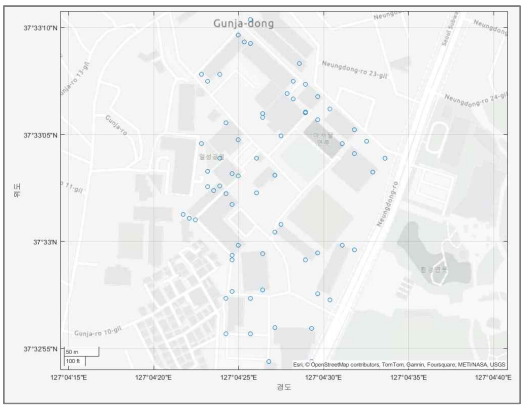


그림 35 건물 경계 위치

```
% course 별 GPS 데이터를 통해  코스별 위치한 건물 번호를 반환
% 중요 파라미터
% vel_min:  실제로 움직인다고 가정하는 최소 속도
vel_min = 0.8;

% 속도가 최소 속도 이상일때 카운트 증가 (c = c + 1), c > count_min 일때 GPS 데이터 재지정
count_min = 5;

% 건물 경계 위치 배열 (위도, 경도)
% 1 충무관, 2 다산관, 3 학술정보원, 4 영실관, 5 우정당, 6 박물관, 7 새날관, 8 용덕관, 9 진관홀
% 10 광개토관, 11 세종관, 12 군자관, 13 학생회관, 14 집현관, 15 대양홀, 16 이당관, 17 애지헌
chungmu = [37.55166, 127.0740;37.55208, 127.0745; 37.55257, 127.0738; 37.55217, 127.0733];
dasan = [37.55208, 127.0745;37.55257, 127.0738; 37.55288 127.0738; 37.55231, 127.0746];
hakjung = [37.55137, 127.0743; 37.55167, 127.0747; 37.55192, 127.0744; 37.55161, 127.0740];
young = [37.55208, 127.0731; 37.55259, 127.0737; 37.55268, 127.0736; 37.55217, 127.0730];
ujung = [37.55168, 127.0747; 37.55188, 127.0749; 37.55204, 127.0747; 37.55185, 127.0745];
museum = [37.55127, 127.0753; 37.55145, 127.0755; 37.55172, 127.0751; 37.55158, 127.0749];
saenal = [37.55090, 127.0758; 37.55108, 127.0760; 37.55130, 127.0757; 37.55114, 127.0755];
yongduk = [37.55108, 127.0733; 37.55132, 127.0736; 37.55154, 127.0734; 37.55127, 127.0730];
jingwan = [37.55072, 127.0733; 37.55088, 127.0735; 37.55108, 127.0733; 37.55091, 127.0731];
gwang = [37.54982, 127.0735; 37.54995, 127.0736; 37.55048, 127.0735; 37.55062, 127.0734;37.55028, 127.0729 ];
sejong = [37.54976, 127.0747; 37.54985, 127.0749; 37.55022, 127.0743; 37.55012, 127.0742];
gunja = [37.54935, 127.0735; 37.54937, 127.0740; 37.54984, 127.0740; 37.54976, 127.0735];
hakgwan = [37.54924, 127.0751; 37.54989, 127.0755; 37.54995, 127.0753; 37.54932, 127.0749];
jiphyun = [37.54880, 127.0734; 37.54880, 127.0738; 37.54926, 127.0738; 37.54926, 127.0734];
daeyang = [37.54844, 127.0741; 37.54844, 127.0748; 37.54887, 127.0748; 37.54888, 127.0742];
edang = [37.55030, 127.0728; 37.55066, 127.0732; 37.55071, 127.0731; 37.55035, 127.0727];
aeji = [37.55063, 127.0739; 37.55086, 127.0742; 37.55108, 127.0739; 37.55085, 127.0736];
```

그림 36 건물 경계

또한 inpolygon 함수를 사용하여 위치 데이터가 건물 경곗값에 포함되는 빈도수를 측정하였다. inpolygon 함수는 기준점에서 수평으로 선을 그어 그러진 경계와 만나는 횟수를 측정하는 방법을 사용한다. 경계와 만나는 횟수가 짝수이면 논리값 0을 홀수이면 논리값 1을 반환한다. 아래는 inpolygon 함수에 대한 참고 자료이다. [2]

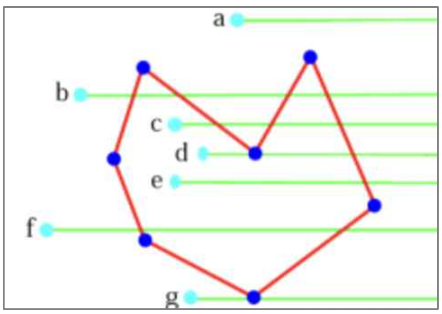


그림 37 그림 예시

```
>> help inpolygon
 inpolygon - 다각형 영역의 내부 또는 모서리에 있는 점
    xq와 yq로 지정된 쿼리 점이 xv와 yv로 정의된 다각형 영역의 내부 또는 모서리에 있는지
    여부를 나타내는 in을 반환합니다.

    구문
      in = inpolygon(xq,yq,xv,yv)
      [in,on] = inpolygon(xq,yq,xv,yv)

    입력 인수
      xq - 쿼리 점의 x 좌표
        스칼라 | 벡터 | 행렬 | 다차원 배열
      yq - 쿼리 점의 y 좌표
        스칼라 | 벡터 | 행렬 | 다차원 배열
      xv - 다각형 꼭짓점의 x 좌표
        벡터
      yv - 다각형 꼭짓점의 y 좌표
        벡터

    출력 인수
      in - 다각형 영역의 내부나 모서리에 있는 점에 대한 표시자
        논리형 배열
      on - 다각형 영역의 모서리에 있는 점에 대한 표시자
        논리형 배열

    비고
      볼록 다각형 내부에 있는 점
      다중연결 다각형 내부에 있는 점
      자기교차 다각형의 내부에 있는 점

    참고 함수 delaunay

    R2006a 이전에 MATLAB에서 개발됨
    inpolygon에 대한 문서
```

그림 38 inpolygon 함수 설명

2.3.5.b GNSS 데이터 오차 보정 과정

*converter 함수

스마트폰의 위치 데이터 수집하기 위해 GNSS 기반 위치 정보를 트래킹해 주는 어플리케이션을 사용하였다. 애플리케이션을 통해 받아온 위치 데이터는 CSV 파일 형태이고 이는 time, latitude, longitude, altitude, speed, horizental_accelation, verical_accelation, related altitude data를 셀 형식으로 나타내준다. converter 함수는 애플리케이션을 통해 받아온 위치 데이터를 년, 월, 일, 시, 분, 위도, 경도, 속도 배열로 만들어주는 함수이다. 시간 정보를 한국 표준시로 변환하며, 24시를 넘어가는 데이터가 존재할 경우 시간을 조정한다. 이는 CSV 형태의 파일이름을 입력받아 readtable(filename) 함수를 통해 필요한 데이터를 추출하고 이를 데이터배열(data_array)에 반환하는 형태이다.

```
function data_array = converter(filename)

% csv 파일 읽기
data = readtable(filename);
data = data(:,2:6);  % csv 파일에서 필요한 데이터 추출

% 시간, 위치 정보 저장할 빈 배열 생성
data_array = [];

% 시간, 2차원 위치 데이터 정보 배열 저장
for i=1:numel(data.time)
    time = strsplit(data.time{i},'T');
    dayy = strsplit(time{1},'-');
    dayy = str2double(dayy);
    year = dayy(1); month = dayy(2); day = dayy(3);
    timee = strsplit(time{2},':');
    timee = str2double(timee);

    % 한국 표준시로 변환
    hour = timee(1)+9;
    minute = timee(2);

    % 24시를 넘어가는 경우 다음날로 설정
    if hour > 24
        day = day + 1;
        hour = hour - 24;
    end

    lat = data.latitude(i);
    long = data.longitude(i);
    vel = data.speed(i);

    data_set = [year month day hour minute lat long vel];
    data_array = [data_array ; data_set];

end

% 년도 데이터 삭제
data_array(:,1) = [];

end
```

그림 39 CSV 변환

*creat_course 함수

creat_course 함수는 converter 함수를 통해 받아온 위치 및 시간, 속도 데이터를 통해 시간표를 만들어 주는 함수이다. GNSS 정보를 바탕으로 시간표의 큰 틀을 잡고(converter 함수 이용), 오차를 고려한 데이터 수정을 통해 시간표를 확정 짓고, 해당 시간표의 위치 데이터를 바탕으로 건물 번호를 각 시간표에 저장한다.

CSV 데이터화 후 이는 초기화된 현재위치에서 위치가 어느 정도 벗어나면 현재위치를 초기화하고 위치가 벗어나기 전까지의 시간을 측정하여 시작 시간, 끝 시간, 평균 위도 경도값, 측정되는 데이터의 인덱스 범위를 course 배열로 반환해 주는 형태이다. 위치의 벗어남 정도를 위도, 경도 차 0.0002도로 설정하였고 최소 데이터의 인덱스 범위를 600으로 설정하였다. 이는 대략 10분 정도이다.

위성 항법 기반 위치 데이터에는 GNSS 신호 오차에 의해 급격하게 오차가 증가하는 데이터튐 현상이 발생한다. 따라서 위치가 벗어났을 때를 카운트하였고, 이는 위치 데이터 오차가 급격하게 튀는 현상을 방지해 준다. 즉, 위치 유지 시간이 약 10분 이상이며, 데이터 튐 10회 까지는 하나의 course로 생각하여 해당 시간 범위를 시간표로 반환한다. 또한 위치 유지 기간의 위도, 경도를 1차 곡선으로 fitting 하여 위도, 경도의 평균값을 구함으로써 위치 오차를 안정시킬 수 있다. 아래의 사진은 11월19일 학술정보원에서의 GNSS 데이터를 추적한 모습이다.

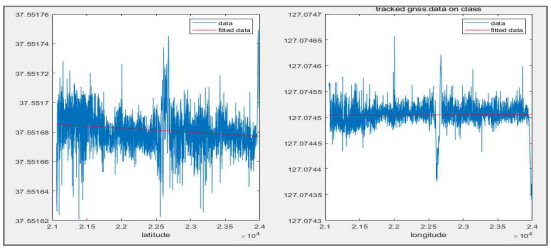


그림 40 학술정보원에서의 GNSS 추적 결과

```
% 파라미터 place_error 위치 벗어남 정도
place_error = 0.00015;

% 시간표 저장
for i = 1:length(data)
    t = t+1;

    % 위치 벗어남, 카운트 늘림 (c = c + 1)
    if abs(data(i,5)-currentplace(1,1))>place_error || abs(data(i,6)-currentplace(1,2)) > place_error
        c = c+1;
    end

    % 위치 유지 시간 600 (약 10분)이상이고 위치벗어났을때
    % lat,long = 유지 시간동안 위도 경도값 행렬
    % 1차 곡선으로 fitting 하여 오차를 줄인 다음 평균값을 course에 저장
    if t > 600  % 유지 시간 약 10분 초과
        if c > 10 % 위치 벗어남
            et = [data(i,3) data(i,4)];
            x = i-t:i;
            lat = data(x,5)';
            long = data(x,6)';
            p1 = polyfit(x,lat,1);
            p2 = polyfit(x,long,1);
            fit_lat = polyval(p1,x);
            fit_long = polyval(p2,x);

            % course에 시작시간 끝시간 위도 경도 평균값 데이터 인덱스 시작,끝 넣버 입력
            course = [course; st et mean(fit_lat) mean(fit_long) i-t i];
            currentplace = [data(i,5) data(i,6)];

            % 시작 시간 및 카운트 수 초기화
            st = [data(i,3) data(i,4)];
            t = 0;
            c = 0;
        end

    % 유지시간 600보다 작고 위치 벗어났을때 현재위치 초기화 및 시간 초기화
    elseif c > 10  % 위치 벗어남
        currentplace = [data(i,5) data(i,6)];
        st = [data(i,3) data(i,4)];
        t = 0;
        c = 0;
    end

end
```

그림 41 GPS 정보를 바탕으로 시간표 1차 저장

시간표 변환 과정에서 위치 오차와 사용자의 이동에 의해 시간표가 끊이는 문제, 하나의 시간표가 여러 행으로 구별되는 문제가 발생하였고 이를 30분 기준으로 시간표를 통합하여 해결하였다. 아래 사진과 같이 11월 18일과 11월 19일의 course 데이터와 평균 위도, 경도값을 geoplot 함수를 통해 지도에 표시하면 아래와 같다.

course

6x9 double

1	2	3	4	5	6	7	8	9
10	26	11	45	37.5520	127.0742	502	4658	1
11	54	13	22	37.5520	127.0734	5087	10315	1
13	23	14	49	37.5524	127.0741	10372	15165	2
15	24	16	9	37.5519	127.0734	17070	19679	1
16	10	16	22	37.5524	127.0741	19770	20535	2
16	36	21	45	37.5517	127.0745	21059	23991	3

그림 42 초기 course 결과값

```
% GPS 오차에 의해 같은 클래스가 여러번 반환되었을때 이를 합쳐주는 코드
% course 위도 경도값 error < 0.0002 보다 작고, 시간 차이 30분보다 작을 때 둘을 합침

over_course = [];
if length(course) > 0
for i = 2:length(course(:,1))
    if (abs(course(i,5)-course(i-1,5)) < 0.0002 && abs(course(i,6)-course(i-1,6)) < 0.0002 && ...
            abs((course(i,1)*60+course(i,2))-(course(i-1,3)*60+course(i-1,4))) < 30)
        over_course = [over_course,i-1];  % 합쳐진 인덱스 저장

        % i번째와 i-1번째가 같으므로 i번째에 i-1번째 데이터 저장
        course(i,1) = course(i-1,1);
        course(i,2) = course(i-1,2);
        course(i,7) = course(i-1,7);
        course(i,5) = (course(i,5)+course(i-1,5))/2;
        course(i,6) = (course(i,6)+course(i-1,6))/2;
    end
end

end

% 중복 데이터 삭제
course(over_course,:)=[];
```

그림 43 여러 시간표 30분 기준 통합

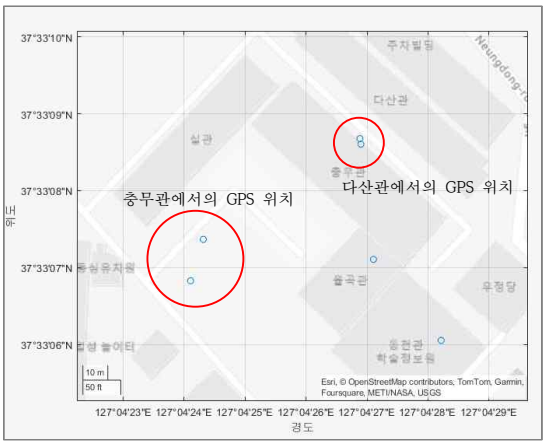


그림 43 데이터 샘플 1

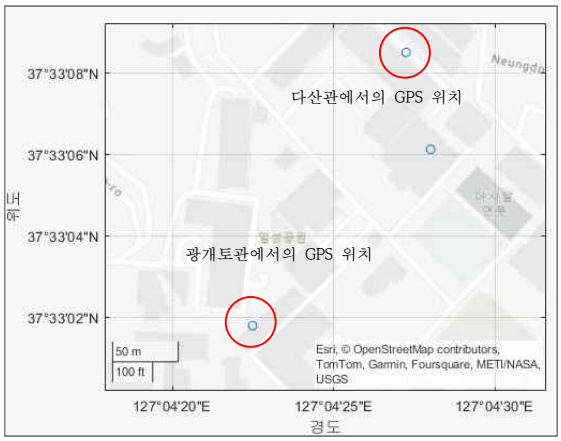


그림 44 데이터 샘플 2

그림 49, 50은 위의 과정을 거쳐 얻은 시간표의 위치 점을 지도에 나타낸 사진이다. 은 충무관 주차장 쪽 두 점은 충무관에서의 수업 시간 동안을, 충무관과 다산관 사이의 두 점은 다산관에서의 위치 정보이다. 또 이당관 위의 점은 광개토관에서의 위치 정보이다. 지도와 GNSS 정보와의 차이점을 고려해도 충무관과 광개토관에서의 위치오차가 상당히 큼을 알 수 있다.

이러한 위치 오차를 고려하여 각 코스에 맞는 건물을 할당한다. 몇몇 건물에서는 위성 신호가 약하거나 수신하는 신호의 양이 적어 상당히 큰 위치오차가 발생한다. 이때 관성 항법에 의한 위치추적에 의존하게 되고, 이는 점점 오차가 증가하는 문제를 야기한다. 이로 인해 위성 신호가 약한 건물에서 위치를 추적하게 되면 잘못된 위치 주위를 맴도는 현상이 발생한다. (그림 45 참고)

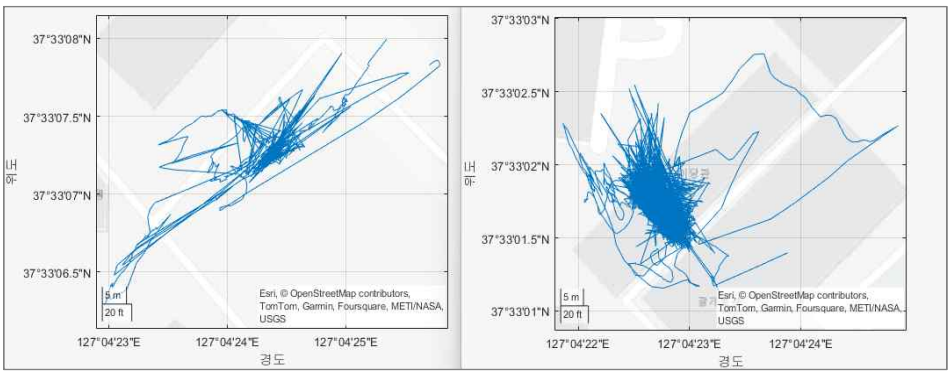


그림 46 실내에서 측정 시 위치 오차 발생

이러한 현상을 해결하기 위해 GNSS 동적 측정 현상을 이용하였다. GNSS 동적 측정이란 기기가 신호를 측정할 때 동일한 위치에 머물러 측정하는 것보다 움직이면서 측정할 때 더 정확하게 측정되는 것이다. 이는 위성 신호를 다양한 각도에서 받을 수 있을 뿐 아니라 시간에 따른 위치 차이를 계산하여 이를 위치보정에 반영할 수 있기 때문이다. 따라서 우리는 실내에서 측정된 잘못된 위치가 움직이기 시작할 때 점차 수정됨을 확인할 수 있다. 아래 사진은 시간표 별로 속도가 1 이상일 때부터 인덱스 범위 100개의 데이터를 트래킹한 모습이다.

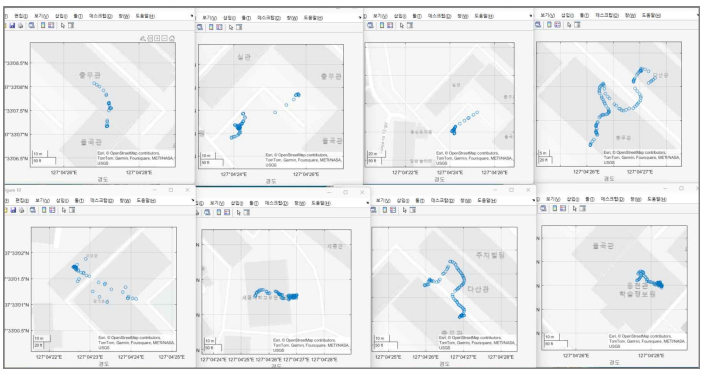


그림 47 데이터 트래킹 예시

모든 사진에서 움직이기 시작하고 건물을 나오기 전까지 위치오차가 계속 수정됨을 볼 수 있다. 또한 위성 신호가 약한 건물에서는 관성 항법에 의한 오차가 계속 누적되므로 위치 데이터의 표준편차가 위성 신호 오차가 적은 건물에 비해서 큼을 확인할 수 있었다. 건물 할당 과정에서는 이러한 현상을 이용해 실내 위치오차를 보정하고 이를 통해 시간표별 건물명을 반환해 준다.

우선 위치오차에 의한 속도 값이 대체로 1보다 낮음을 반영하여 최소 속력 파라미터값과 최소 표준편차 파라미터값을 1, 0.00005로 설정하였다. 시간표별 위치 데이터의 표준

편차가 설정값보다 낮을 때는 위성 신호 수신이 원활함을 의미한다. 즉 해당 시간표의 속력 표준편차가 0.00005 이내면 한 자리에 잘 멈춰 있다고 판단, 건물 빈도 체크 과정을 거치고, 표준편차가 0.00005 이상인 경우 한 자리에 잘 멈춰있지 않다고 판단, 시간표의 인덱스 (전체 변환한 데이터 기준)의 범위를 조정한다. 조정 조건은 속력이 약 1 (코드에서는 더 정밀히 오차를 잡기 위해 0.8로 설정)보다 작은 경우, 멈춰 있는 경우로 보되, 최소 속력을 초과하는 횟수가 5회를 초과하기 전까지를 강의실에 멈춰 있는 기준으로 삼는다. 표준편차가 기준을 넘어가면서, 속력을 고려하여 오차를 5회까지 고려할 때, 전체 데이터 범위에서 시간표의 인덱스 범위를 해당 인덱스부터 -30까지의 인덱스를 새로운 위치 데이터 인덱스 범위로 설정한다. 조정한 범위 내에서 위도와 경도를 배열로 저장한다. 조정된 범위 내에서 순 위치 데이터값이 건물 경계 내에 포함되는 빈도수를 측정하여 건물명을 반환해 준다. (건물 번호는 초기 분류된 시간표에 할당) 이 과정에서 inpolygon 함수를 사용하였다. 아래 사진은 inpolygon 함수를 사용하여 건물 경계 내부에 있는 위치 데이터값을 외부에 있는 위치 데이터값과 비교하여 그린 모습이다. (그림 45, 빨간 부분은 건물 내부, 파란 부분은 건물 외부 의미)

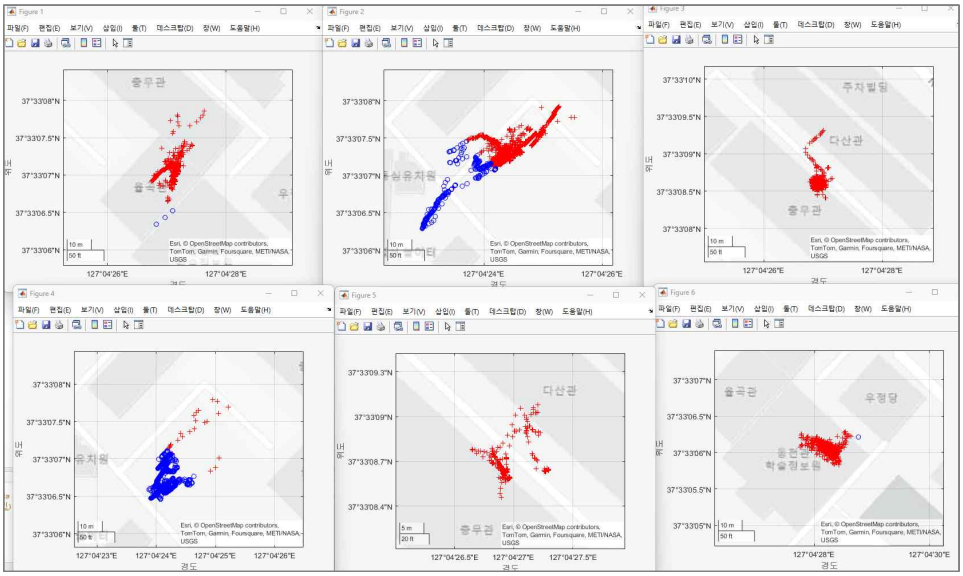


그림 48 데이터 비교 그래프

각 시간표들의 데이터가 건물별 경계에 접하여 빈도수가 가장 큰 건물의 번호를 반환된다. 이는 course 배열의 9열에 입력된다.

아래 사진은 11월 18일(월), 19일(화), 20일(수)의 데이터를 입력하고 course_mon, course_tues, course_wens 로 반환해 본 모습이다. 건물번호 1번은 충무관, 2번은 다산관, 3번은 학술정보원, 10번은 광개토관, 12번은 군자관을 의미한다. (표 1 참고) 이는 5분에서 20분 내외의 수업 시간 오차를 가지지만 건물명은 실제 시간표와 일치하였다.

course_mon

2x9 double

1	2	3	4	5	6	7	8	9
15	4	16	15	37.5496	127.0740	1	4067	12
16	19	18	44	37.5505	127.0729	4280	12908	10

그림 49 11월 18일 월요일 데이터 예시

course_mon | course_tues

5x9 double

1	2	3	4	5	6	7	8	9
10	25	11	40	37.5520	127.0742	461	4366	1
11	54	12	50	37.5520	127.0735	5102	8400	1
13	25	14	49	37.5524	127.0741	10446	15121	2
15	34	15	55	37.5519	127.0734	17655	18827	1
16	36	21	45	37.5517	127.0745	21054	23972	3

그림 50 11월 19일 데이터 예시

course_mon | course_tues | course_wens

3x9 double

1	2	3	4	5	6	7	8	9
16	49	19	37	37.5505	127.0730	1400	9679	10
20	0	20	50	37.5524	127.0742	11192	13724	2
21	21	24	51	37.5517	127.0744	14581	16528	3

그림 51 11월 20일 데이터 예시

#	건물이름	#	건물이름	#	건물이름
1	충무관	7	새날관	13	학생회관
2	다산관	8	용덕관	14	집현관
3	학술정보원	9	진관홀	15	대양홀
4	영실관	10	광개토관	16	이당관
5	우정당	11	세종관	17	애지헌
6	박물관	12	군자관		

표 1 건물 할당 번호

```
% 건물 탐색
for i = 1:length(course(:,1))
    % contact하는 건물 배열 생성
    build_contact = zeros(17,1);

    % 건물 번호 체크를 위한 초기값 설정
    max = 0;
    build_num = 0;

    % course 위도,경도 데이터 표준편차 0.00005 이하일 경우
    % GPS 좌표가 건물별 경계에 포함되는 빈도수 측정
    if std(data(course(i,7):course(i,8),5)) < 0.00005 && std(data(course(i,7):course(i,8),6)) < 0.00005

        % 건물별 contact 빈도수 체크
        for j=course(i,7):course(i,8)
            if inpolygon(data(j,5),data(j,6),chungmu(:,1),chungmu(:,2))
                build_contact(1,1) = build_contact(1,1)+1;
            elseif inpolygon(data(j,5),data(j,6),dasan(:,1),dasan(:,2))
                build_contact(2,1) = build_contact(2,1)+1;
            elseif inpolygon(data(j,5),data(j,6),hakjung(:,1),hakjung(:,2))
                build_contact(3,1) = build_contact(3,1)+1;
            elseif inpolygon(data(j,5),data(j,6),young(:,1),young(:,2))
                build_contact(4,1) = build_contact(4,1)+1;
            elseif inpolygon(data(j,5),data(j,6),ujung(:,1),ujung(:,2))
                build_contact(5,1) = build_contact(5,1)+1;
            elseif inpolygon(data(j,5),data(j,6),museum(:,1),museum(:,2))
                build_contact(6,1) = build_contact(6,1)+1;
            elseif inpolygon(data(j,5),data(j,6),saenal(:,1),saenal(:,2))
                build_contact(7,1) = build_contact(7,1)+1;
            elseif inpolygon(data(j,5),data(j,6),yongduk(:,1),yongduk(:,2))
                build_contact(8,1) = build_contact(8,1)+1;
            elseif inpolygon(data(j,5),data(j,6),jingwan(:,1),jingwan(:,2))
                build_contact(9,1) = build_contact(9,1)+1;
            elseif inpolygon(data(j,5),data(j,6),gwang(:,1),gwang(:,2))
                build_contact(10,1) = build_contact(10,1)+1;
            elseif inpolygon(data(j,5),data(j,6),sejong(:,1),sejong(:,2))
                build_contact(11,1) = build_contact(11,1)+1;
            elseif inpolygon(data(j,5),data(j,6),gunja(:,1),gunja(:,2))
                build_contact(12,1) = build_contact(12,1)+1;
            elseif inpolygon(data(j,5),data(j,6),hakgwan(:,1),hakgwan(:,2))
                build_contact(13,1) = build_contact(13,1)+1;
            elseif inpolygon(data(j,5),data(j,6),jiphyun(:,1),jiphyun(:,2))
                build_contact(14,1) = build_contact(14,1)+1;
            elseif inpolygon(data(j,5),data(j,6),daeyang(:,1),daeyang(:,2))
                build_contact(15,1) = build_contact(15,1)+1;
            elseif inpolygon(data(j,5),data(j,6),edang(:,1),edang(:,2))
                build_contact(16,1) = build_contact(16,1)+1;
            elseif inpolygon(data(j,5),data(j,6),aeji(:,1),aeji(:,2))
                build_contact(17,1) = build_contact(17,1)+1;
            end
        end
```

그림 52 표준편차가 범위 내인 경우

```
        % 표준편차 0.00005 이상일 경우
        % course 별 끝 인덱스 번호 -60 부터 속도가 vel_min 이상일 때 카운트 늘림
        % 카운트 5 이상일 때의 인덱스 넘버(end_num)부터 end_num-30 (start_num) 범위 지정
        % start_num 부터 end_num 범위의 GPS 좌표 재지정 re_lat, re_long
        % re_lat, re_long의 GPS 데이터가 건물별 바운더리에 포함되는 빈도수 측정

    else
        % 초기값 설정
        c = 0;
        re_lat = [];
        re_long = [];
        j = course(i,8)-60:length(data);

        if course(i,8) < 102
            j = 2:length(data);
        end

        for j = j
            if data(j,7) > vel_min
                c = c+1;
            end
            if c > count_min
                end_num = j;
                start_num = j-30;
                if start_num < 1
                    start_num = 1;
                end
                re_lat = data(start_num:end_num,5);
                re_long = data(start_num:end_num,6);
                break
            end
        end

        for j = 1:length(re_lat)
            if inpolygon(re_lat(j,1),re_long(j,1),chungmu(:,1),chungmu(:,2))
                    build_contact(1,1) = build_contact(1,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),dasan(:,1),dasan(:,2))
                    build_contact(2,1) = build_contact(2,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),hakjung(:,1),hakjung(:,2))
                    build_contact(3,1) = build_contact(3,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),young(:,1),young(:,2))
                    build_contact(4,1) = build_contact(4,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),ujung(:,1),ujung(:,2))
                    build_contact(5,1) = build_contact(5,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),museum(:,1),museum(:,2))
                    build_contact(6,1) = build_contact(6,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),saenal(:,1),saenal(:,2))
                    build_contact(7,1) = build_contact(7,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),yongduk(:,1),yongduk(:,2))
                    build_contact(8,1) = build_contact(8,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),jingwan(:,1),jingwan(:,2))
                    build_contact(9,1) = build_contact(9,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),gwang(:,1),gwang(:,2))
                    build_contact(10,1) = build_contact(10,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),sejong(:,1),sejong(:,2))
                    build_contact(11,1) = build_contact(11,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),gunja(:,1),gunja(:,2))
                    build_contact(12,1) = build_contact(12,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),hakgwan(:,1),hakgwan(:,2))
                    build_contact(13,1) = build_contact(13,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),jiphyun(:,1),jiphyun(:,2))
                    build_contact(14,1) = build_contact(14,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),daeyang(:,1),daeyang(:,2))
                    build_contact(15,1) = build_contact(15,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),edang(:,1),edang(:,2))
                    build_contact(16,1) = build_contact(16,1)+1;
                elseif inpolygon(re_lat(j,1),re_long(j,1),aeji(:,1),aeji(:,2))
                    build_contact(17,1) = build_contact(17,1)+1;
            end
        end
```

그림 53 표준편차 범위 밖인 경우

```
% 건물별 최고 빈도수 비교 및 build_name, course(i,9)에 입력
for j = 1:length(build_contact)
    if build_contact(j,1) > max
        max = build_contact(j,1);
        build_num = j;
    end
end
course(i,9) = build_num;
```

그림 54 건물 번호 각 시간표에 할당

2.3.5.c 시간표 데이터화

***coursetotimetable 함수**

coursetotimetable 함수는 creatcourse 함수로 만들어진 코스 배열을 입력받아 시간표 structure로 반환하는 함수이다. 이때 수업 시간을 1시간 30분 기준으로 수업 시간표에 맞는 시작 시간, 끝 시간 건물번호가 timetable 구조에 저장된다.

```
%%%
```

2.3.5.d 시간표 형성

시간표 GUI 는 3가지 버튼으로 구성된다. 왼쪽의 '시간표 업로드' 버튼을 누르면 연결된 구글 드라이브 폴더로 이동하여, '학번_요일.csv' 형식의 tracking CSV 데이터 파일을 업로드 한다. 해당 버튼은 구글 드라이브 계정주의 환경에서 제대로 작동된다. 이는 링크는 여러 사용자가 동시에 사용할 수 있도록 구글 드라이브로 연동이 되어 있지만 구글 드라이브의 공유 폴더가 폴더 계정 주 외의 경우 로컬과 연동이 안 되는 문제를 발견, 로컬에서 별도로 돌릴 수 있도록 변경하였다. 로컬에서 프로그램을 돌릴 때는, 프로그램 파일 내의 'upload' 폴더에 해당 CSV 파일들을 업로드 한다. (첨부한 코드들은 로컬에서 돌리는 것을 기준으로 작성되었으며, 구글 드라이브로 바로 연동되게 하기 위해서는 폴더의 경로를 수정하는 방식으로 연동할 수 있음.)

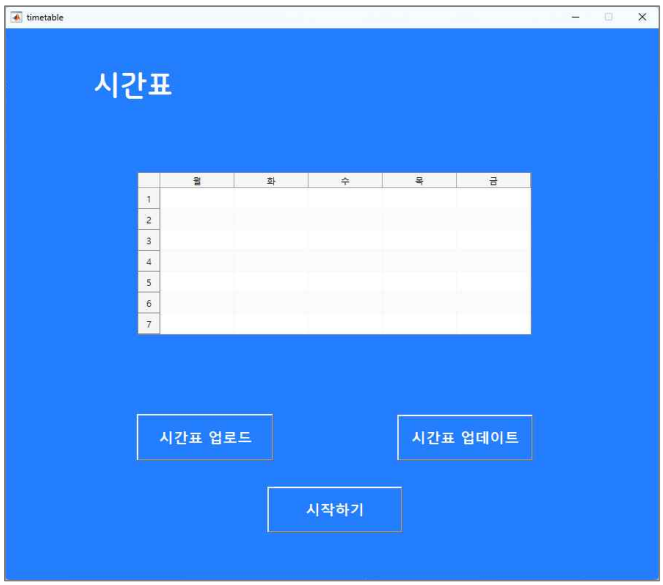


그림 57 시간표 화면

```
% --- Executes on button press in tt_update.
function tt_update_Callback(hObject, eventdata, handles)
    url = 'https://drive.google.com/drive/folders/1-RkRmTVojiyYxKWEizthRnvnTj9ZomYH?usp=sharing'; % 시간표 업로드 URL
    web(url, '-browser'); % 기본 브라우저에서 열기
end
```

그림 58 구글 드라이브 연동

오른쪽의 '시간표 업데이트' 버튼을 누르면, 'upload' 폴더 내의 CSV파일들을 탐색, 분류한다. 정해진 형식의 이름을 가진 파일을 업로드하기에, 해당 폴더에서 '_' 가 들어간 파일들을 탐색하면 tracking 데이터 파일이 탐색 된다. '_' 를 기준으로 앞이 학번, 뒤가 요일로 저장되며, 'usertracks' 폴더에 파일의 학번으로 된 폴더가 생성되며, 학번 폴더 안에 그 학번 회원의 CSV tracking 데이터 파일이 이동된다. CSV 파일이 이동되면, 해당 데이터를 시간표화 한다. 'creatcourse' 함수, 'coursetotimetable' 함수, 'timetabletotext' 함수를 차례로 실행하여 최종적으로 탐색한 회원의 CSV 데이터가 해당 요일의 전체 시간표를 건물번호로 반환하게 한다. 수업이 없으면 0, 수업이 있으면 수업이 있던 건물의 번호를 저장한다. 배열로서 저장된 timetext를 차례로 전체 회원의 시간표 정보가 담겨있는 user_timetable.txt 파일에 새로 저장한다. (그림 59)

이때 기존에 같은 요일의 CSV 파일이 존재할 수 있으므로 전체 user_timetable.txt 파일을 탐색하여 학번+요일이 중복되어 존재하는 경우 가장 최신 저장된 정보를 제외하고 기존에 저장된 시간표 데이터를 삭제한다. 이는 전체 데이터를 구조화하고, 중복된 값을 제거한 전체 데이터를 txt 파일에 다시 작성하는 방식으로 이루어진다. (그림 60)

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%  user_timetable.m
%
%  Users가 특정 폴더에 업로드 한 GPS tracking data(.csv)를 시간표로 변환
%   1. 새 csv 파일 데이터 txt 파일에 저장
%   2. 중복되는 데이터 (학번+요일 동일)의 경우 최신 데이터로 업데이트
%   3. 저장한 csv 파일 유저 별 폴더로 분류
%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

h=waitbar(0,'Updating....(잠시만 기다려주세요)');

%   1. 새 csv 파일 데이터 txt 파일에 저장


% 현재 디렉토리에서 파일 생성
filename = fullfile('user_timetable.txt');

% CSV 파일 목록 가져오기
usertracks = 'upload\';
files = dir(fullfile(usertracks, '*.csv'));

disp('찾은 CSV 파일 목록:');
disp({files.name}); % CSV 파일 목록 출력

% 파일 열기 (append 모드)
fid = fopen(filename, 'at');
if fid == -1
    error('user_timetable.txt 파일을 열 수 없습니다. 경로와 권한을 확인하세요.');
end

% 각 CSV 파일 처리
for i = 1:length(files)
    % 파일명 가져오기
    filenames = files(i).name;

    % 파일명의 '_' 위치 찾기
    underindex = find(filenames == '_', 1);

    % 잘못된 파일 형식 무시
    if isempty(underindex)
        disp(['잘못된 파일 형식: ', filenames]);
        continue;
    end

    % 학번과 요일 추출
    id = filenames(1:underindex-1);
    day = filenames(underindex+1:underindex+3);

    disp(['처리 중인 파일: ', filenames]);
    disp(['추출된 학번: ', id, ', 추출된 요일: ', day]);

    % 학번별 폴더 생성
    folder_path = fullfile('usertracks\', id); % 학번 폴더 경로
    if ~exist(folder_path, 'dir')
        mkdir(folder_path); % 폴더가 없으면 생성
        disp(['폴더 생성: ', folder_path]);
    end

    % CSV 파일을 해당 학번 폴더로 이동
    new_file_path = fullfile(folder_path, filenames); % 새 경로
    movefile(fullfile(usertracks, filenames), new_file_path);
    disp(['파일 이동: ', filenames, ' -> ', folder_path]);

    % 시간표 데이터 생성
    try
        % 새 경로에서 파일 처리
        course = creatcourse(new_file_path); % 코스 데이터 생성
        timetable = coursetotimetable(course); % 시간표 데이터 변환
        timetext = timetabletotext(timetable); % 텍스트 시간표 변환

        % 데이터가 올바른지 확인
        if length(timetext) < 7
            warning('timetext 데이터가 충분하지 않습니다: %s', filenames);
            continue;
        end

        % user_timetable.txt에 데이터 저장
        fprintf(fid, '%s\t%s\t%d\t%d\t%d\t%d\t%d\t%d\n', id, day, timetext(1:7));
        disp(['저장된 데이터: ', sprintf('%s\t%s\t%d\t%d\t%d\t%d\t%d\t%d\t%d', id, day, timetext(1:7))]);
    catch ME
        warning('파일 %s 처리 중 오류 발생: %s', new_file_path, ME.message);
    end
end

% 파일 닫기

fclose(fid);
disp('user_timetable.txt 파일 생성 완료.');
```

그림 59 CSV 탐색 및 시간표화, 저장

```
%   2. 중복되는 데이터 (학번+요일 동일)의 경우 최신 데이터로 업데이트


% 중복 삭제
fid = fopen(filename,'rt');
i = 1;
idlist = {};
usertable={};
while ~feof(fid)
    line = fgetl(fid);
    if line == -1
        break
    end

    linedata = strsplit(line,'\t');
    usertable(i).id = linedata(1);
    usertable(i).day = linedata(2);
    usertable(i).table = linedata(3:9);
    % 학번, 요일이 같은 경우 비교를 위해 문자열로 저장
    usertable(i).iday = sprintf('%s%s',usertable(i).id,usertable(i).day);
    % 학번, 요일이 같은 데이터인 경우 해당하는 index 추가
    if ~ismember(usertable(i).iday,idlist)
        idlist{end+1} =  usertable(i).iday;
    end
    i = i +1;
end

for j = 1:length(idlist)
    index = find(strcmp({usertable.iday},idlist(j)));
    index = index(1:end-1);
    usertable(index) = []; % 중복되는 index 중 마지막(최신 데이터) 를 제외하고 삭제
end

fid = fopen(filename,'wt');
% 새 시간표 파일 작성
for k = 1:length(usertable)
    fprintf(fid,'%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n',usertable(k).id,usertable(k).day,...
        usertable(k).table{1},usertable(k).table{2}, usertable(k).table{3}, ...
        usertable(k).table{4}, usertable(k).table{5}, usertable(k).table{6},usertable(k).table{7});
end

fclose(fid);
```

그림 60 중복 삭제

이후 사용자의 시간표 정보를 GUI에 보여주기 위해서 user_data.txt 파일에 저장된 사용자의 학번을 이용하여 user_timetable.txt 파일에서 해당 사용자 학번의 시간표를 탐색하여 교시별로 저장한다. 수업이 없어 번호가 0인 경우, 빈칸으로 저장하며, 수업이 있는 경우 번호에 해당하는 건물명을 시간표 데이터에 저장하고, 셀 형식으로 변환한다. 이를 set 함수를 통해 uitable 변수에 입력하면 사용자의 시간표가 만들어지게 된다.

시작 버튼을 누르면 되면 선호 선택을 하는 GUI가 실행된다.

```
% 로그인한 학번 가져오기
userdata_filename = 'user_data.txt';
if exist(userdata_filename, 'file')
    fid = fopen(userdata_filename, 'rt');
    user_lines = textscan(fid, '%s', 'Delimiter', '\n');
    fclose(fid);

    id_line = user_lines{1}{end}; % 마지막 학번 (로그인 학번)
    id_split= strsplit(id_line,'\t');
    id=id_split{1,1};
else
    errordlg('user_data.txt 파일이 없습니다. 먼저 로그인하세요.', '오류');
    return;
end
```

그림 61 사용자의 학번 추출

```
% 강의실 이름
classname = ["충무관", "다산관", "학술정보원", "영실관", "무정당", "박물관", "새날관", "용덕관", ...
             "진관홀", "광개토관", "세종관", "군자관", "학생회관", "집현관", "대강홀", "이양관", "예지헌"];

% 요일 및 시간표 초기화
class = {'1교시', '2교시', '3교시', '4교시', '5교시', '6교시', '7교시'};
mon = strings(7, 1);
tue = strings(7, 1);
wen = strings(7, 1);
thu = strings(7, 1);
fri = strings(7, 1);

% 각 요일에 대한 시간표 설정
for i = 1:length(user_timetable)
    switch user_timetable(i).day
        case 'mon'
            dayArray = mon;
        case 'tue'
            dayArray = tue;
        case 'wen'
            dayArray = wen;
        case 'thu'
            dayArray = thu;
        case 'fri'
            dayArray = fri;
    end

    for j = 1:7
        classNum = user_timetable(i).(['class', num2str(j)]);
        if classNum == 0
            dayArray(j) = '';
        else
            dayArray(j) = classname(classNum);
        end
    end

    switch user_timetable(i).day
        case 'mon'
            mon = dayArray;
        case 'tue'
            tue = dayArray;
        case 'wen'
            wen = dayArray;
        case 'thu'
            thu = dayArray;
        case 'fri'
            fri = dayArray;
    end
end
```

그림 62 시간표 셀 변환

2.3.6 선호 선택 (1)

이 프로그램의 메인 화면에서는 사용자가 설정한 선호 정보와 데이터를 기반으로 만남을 매칭하는 과정을 관리한다. 메인 화면은 GUI를 통해 사용자가 원하는 만남의 유형(밥 친구, 공부 친구, 일반 친구)을 선택할 수 있도록 설계되어 있다. 아래는 해당 코드를 기반으로 한 프로그램 동작 및 기능 설명이다.

```
function meal_Callback(hObject, eventdata, handles)
    update_user_tag('meal');
end


% --- Executes on button press in friend.
function friend_Callback(hObject, eventdata, handles)
    update_user_tag('friend');
end

% --- Executes on button press in study.
function study_Callback(hObject, eventdata, handles)
    update_user_tag('study');
end
```

그림 63 원하는 만남 유형 선택하는 버튼

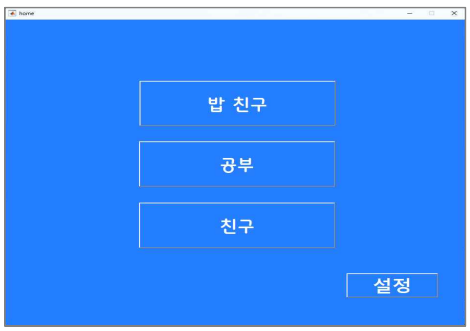


그림 64 메인 화면의 세 가지 선호 선택 버튼

만남의 유형을 선택하는 메인 화면 코드는 만남 유형을 선택하는 세거지 버튼과 설정 버튼 있고, 데이터 처리 부분, 예외 처리 부분으로 구성되어 있다.

먼저 사용자가 '밥 친구', '공부', '친구' 버튼 중 하나를 클릭하면 해당 콜백 함수(meal_Callback, study_Callback, friend_Callback)가 실행된다. 이 함수들은 공통적으로 update_user_tag 함수를 호출하며, 사용자가 선택한 선호 유형을 파일에 저장한다.

데이터를 저장하는 update_user_tag 함수는 user_data.txt 파일에서 현재 로그인한 사용자의 학번을 읽고, 학번과 선호 정보를 tab(\t)으로 구분된 형식인 user_want.txt 파일에 저장하는 함수이다. 파일에서 기존 데이터를 읽어온 뒤, 로그인한 사용자의 데이터를 삭제하여 중복을 방지하고, 중복이 제거된 데이터와 함께 새로운 선호 정보를 파일에 다시 기록한다. 만약, 선호 정보를 저장할 파일이 없으면 새 파일을 생성한다.

```
function update_user_tag(tag)

% 1. 입력값 가져오기
filename = 'user_data.txt';

% 로그인 학번 가져오기
if exist(filename, 'file') % user_data.txt 파일 존재 여부 확인
    fid = fopen(filename, 'rt'); % 읽기 모드로 열기
    stid = strtrim(fgetl(fid)); % 첫 줄에서 학번 읽기
    fclose(fid);
else
    errordlg('user_data.txt 파일이 없습니다. 먼저 로그인하세요.', '오류');
    return;
end

% user_want.txt 파일 처리
filename = 'user_want.txt';

% 파일이 존재하지 않을 경우 빈 파일 생성
if ~isfile(filename)
    fid = fopen(filename, 'wt');
    fclose(fid); % 파일 바로 닫기
end

% 파일 읽기
fid = fopen(filename, 'rt');
user_want = {}; % 데이터 저장할 빈 배열 생성
k = 1;

while ~feof(fid)   % 파일의 끝에 도달하지 않았으면 반복
    line = strtrim(fgetl(fid)); % 현재 줄을 읽고 공백 제거

    % 빈 줄이나 잘못된 줄 무시
    if isempty(line)
        continue;
    end
```

그림 65 선호 유형 저장 1

```
    % 줄을 탭으로 나눔
    linedata = strsplit(line, '\t');

    % 데이터가 학번과 태그 두 개의 요소로 나뉘지 않은 경우 무시
    if numel(linedata) < 2
        continue;
    end

    % user_want 구조체에 데이터 저장
    user_want(k).stid = linedata{1};
    user_want(k).want = linedata{2};
    k = k + 1;  % 다음 user_want 구조체에 저장하기 위해 index 업데이트
end

fclose(fid);

% 중복된 경우 제거
if ~isempty(user_want)
    index = find(strcmp({user_want.stid}, stid));
    user_want(index) = []; % 기존 학번의 데이터 삭제
end

% 파일 다시 쓰기
fid = fopen(filename, "wt");
for j = 1:length(user_want)
    fprintf(fid, '%s\t%s\n', user_want(j).stid, user_want(j).want);
end

% 새 정보 추가 입력
fprintf(fid, '%s\t%s\n', stid, tag);

fclose(fid);

run setting.m;
close(mainhome);
end
```

그림 66 선호 유형 저장 2

메인 화면의 하단에 있는 '설정' 버튼을 누르면, 정보를 수정할 수 있는 modify.m 파일이 실행되어 정보 수정 화면으로 이동한다. 버튼 클릭 시 콜백 함수(modify_Callback):modify.m를 실행하여 메인 화면을 종료하고 정보 수정 화면으로 이동한다. 수정 작업이 완료되면 다시 메인 화면이 나타나게 된다.

예를 들어 사용자가 '밥 친구' 버튼을 클릭하면 meal_Callback 함수가 실행되고, update_user_tag('meal')이 호출된다. user_data.txt 에서 로그인 학번을 읽어온 후, user_want.txt에 저장된다. 사용자가 '설정' 버튼을 클릭하면 modify_Callback 함수가 실행된다. 정보 수정 화면이 실행되고, 메인 화면은 종료된다. 혹 user_data.txt 파일이 없으면 오류 메시지를 출력하고 작업을 중단한다. user_want.txt 가 없는 경우 새 파일을 생성한 뒤 진행된다.

```
function modify_Callback(hObject, eventdata, handles)
run modify.m;
close(mainhome);
% hObject    handle to modify (see GCBO)
% eventdata  reserved - to be defined in a future version of MATLAB
% handles    structure with handles and user data (see GUIDATA)
end
```

그림 67 회원정보 수정

오른쪽 아래의 설정 버튼을 클릭하면 modify.m을 실행하고 현재 메인 화면을 닫는다.

2.3.7 선호 선택 (2)

두 번째 선호 선택 과정은 setting.m을 통해서 이루어진다. 해당 GUI에서 사용자의 선호 정보를 저장할 수 있도록 한다. 이 단계는 크게 4단계로 이루어진다. 입력된 사용자의 선호 정보를 저장하고, 전체 회원의 선호 정보를 구조화하고, 기존 선호 정보가 존재하는지 중복 여부를 파악하여, 최신화된 선호 정보로 저장하는 순서로 이루어진다.

그림 68 setting.fig, '시작'를 누르면 해당 아래의 코드가 돌아감

첫 번째로 ‘user_data.txt’ 파일을 읽기 모드를 통해 사용자의 학번을 저장하고, 해당 GUI 창에 입력한 사용자의 선호 정보를 저장한다. 모든 회원의 선호 정보를 포함하는 ‘user_prefer.txt’ 파일이 존재하지 않는 경우, 새로 만든다. 이후, 읽기 모드를 통해 파일에 저장된 정보를 구조화한다. user_pre 구조에는 선호 정보(학번, 성별, 학과, 학년, 요일)가 stid, gender, major, grade, day의 필드로 저장된다.

```
gender = get(handles.popupmenu_gender, 'String');
gender = gender{get(handles.popupmenu_gender, 'Value')};
major = get(handles.popupmenu_major, 'String');
major = major{get(handles.popupmenu_major, 'Value')};
grade = get(handles.popupmenu_grade, 'String');
grade = grade{get(handles.popupmenu_grade, 'Value')};
day = get(handles.popupmenu_day, 'String');
day = day{get(handles.popupmenu_day, 'Value')};
```

그림 69 figure 입력값 저장

```
filename = 'user_data.txt';
fid = fopen(filename, 'rt');
u = 1;
while ~feof(fid)   % 파일의 끝에 도달하지 않았으면 반복
    line = fgetl(fid);   % 해당 열 문자를 받기

    if line == -1   % 마지막 열일 경우 반복문 종료
        break
    end
    linedata = strsplit(line,'\t');
    user(u).stid = linedata{1}; %user.stid에 각 학번 다 저장
    u = u+1;
end
stid = user(u-1).stid; %user_want.txt의 마지막 행에 있는 학번이 접속한 사람의 학번
```

그림 70 로그인 학번 저장

```
% 2. 데이터 파일에 저장
% Users의 정보를 저장할 txt파일 경로 설정
filename = 'user_prefer.txt';
if ~isfile(filename) %파일이 존재하지 않을경우 빈파일 생성
    fid = fopen(filename, 'wt');
fclose(fid); % 파일 바로 닫기
end
fid = fopen(filename, 'rt');
user_pre = {}; % 데이터 저장할 빈 배열 생성
k = 1;
while ~feof(fid)   % 파일의 끝에 도달하지 않았으면 반복
    line = fgetl(fid);   % 해당 열 문자를 받기

    if line == -1   % 마지막 열일 경우 반복문 종료
        break
    end

    linedata = strsplit(line,'\t');   % User의 정보를 tap 을 기준으로 정보를 쪼개고 user_perfer struct에 저장
    user_pre(k).stid = linedata{1};
    user_pre(k).gender = linedata{2};
    user_pre(k).major = linedata{3};
    user_pre(k).grade = linedata{4};
    user_pre(k).day = linedata{5};

    k = k+1;  % 다음 user struct에 저장하기 위해 index 업데이트
end
```

그림 71 user_prefer.txt 정보 구조화

user_pre 구조의 stid 필드에서 학번이 로그인 학번과 같은 경우에 해당하는 인덱스를 저장한다(user_pre 구조에서 로그인 유저의 기존 선호 정보가 저장되어 있는 순서를 저장한다). 로그인 유저의 기존 정보가 존재할 경우, gui로 입력한 최신화된 정보가 필요하기에, 인덱스를 이용해 user_pre 구조에서 로그인 유저의 기존 선호 정보를 모두 지운다, 로그인 유저의 정보가 user_pre 구조에 존재하지 않으면 인덱스는 배제한다. 파일을 쓰기 모드로 열어 user_prefer에서 추출한 로그인 유저를 제외한 모든 유저의 선호 정보에 수정된 정보를 추가하여 ‘user_prefer.txt’를 재생성한다. 이후 매칭 코드를 실행한다.

```
% 중복된 경우 제거
if length(user_pre)>0
    index = find(strcmp({user_pre.stid},stid));
    user_pre(index) = [];
end
% 다시 새로 작성
fid = fopen(filename,"wt");
for j = 1:length(user_pre)
    fprintf(fid,'%s\t%s\t%s\t%s\t%s\n',user_pre(j).stid,user_pre(j).gender,...
        user_pre(j).major,user_pre(j).grade,user_pre(j).day);
end

% 새 정보 추가 입력
fprintf(fid,'%s\t%s\t%s\t%s\t%s\n',stid,gender,major,grade,day);

fclose(fid);
run matching_gui.m;
close(setting);
```

그림 72 중복 제거 후 파일 재생성

```
user_prefer.txt
23010018    남자    all 4   mon
23010029    all same    all mon
23020018    all other   all mon
23020029    여자    all all mon
23120018    all same    2   tue
23120029    all all all tue
21010514    여자    same    4   tue
21110514    남자    all all tue
23010118    all all all mon
23010129    all other   all mon
23020118    여자    all all mon
23020129    여자    all 4   mon
21010512    남자    same    1   mon
```

그림 73 유저들의 선호 정보 저장 예시

2.3.8 매칭

매칭 과정은 matching_gui.m을 통해서 이루어진다. 해당 GUI에서 유저들의 회원 정보, 선호 정보를 바탕으로 만남이 성사될 수 있도록 한다. 이 단계는 총 3단계로 이루어진다. 로그인 유저의 선호 정보 및 전체 유저 정보를 구조화하고, 전체 유저 정보와 로그인 유저의 선호 정보를 비교하고, 선호 유저 정보와 동일한 유저들 간의 시간표를 비교하는 순서로 이루어진다.

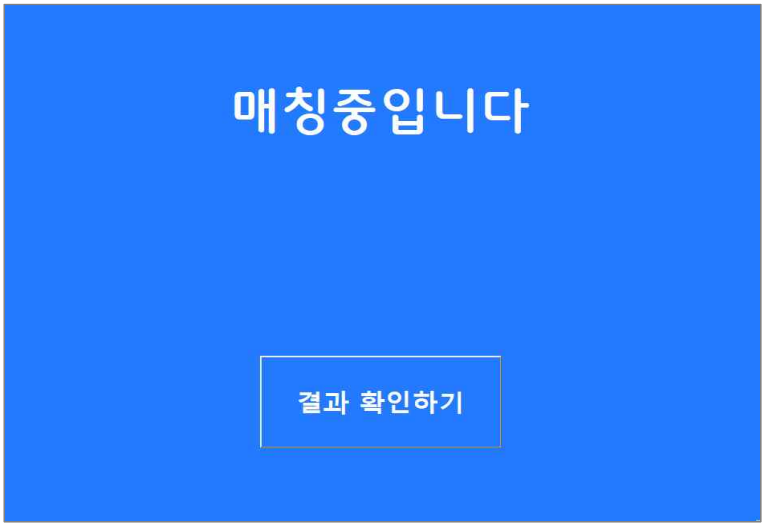


그림 74matching_gui.figure, ‘결과 확인하기’를 누르면 해당 아래의 코드가 돌아감

첫 번째로, 로그인 유저의 선호 정보 및 전체 유저 정보를 구조화하기 위해 데이터베이스 폴더에서 user_want.txt, user_prefer.txt, user.txt, user_timetable.txt를 읽기 모드로 열고 구조화한다. user_pre 구조에는 로그인 한 유저의 정보를 제외한 유저들의 학번과 선호 정보 (user_want, user_prefer)가 id, gender, major, grade, day의 필드로 저장된다. 자신과 매칭되는 것을 방지하기 위해 로그인 한 유저의 선호 정보는 user_mypre 구조에 저장한다. 프로그램을 사용하는 본인을 제외한 전체 유저의 회원 정보들을 user 구조에 id, major, gender, grade, kakao의 필드로 저장한다. 전체 유저의 정보가 담긴 user 구조의 크기와 선호 정보가 담긴 user_pre 구조의 크기가 다른 경우 보정을 위해 usernew 셀에 user_prefer 정보가 있는 유저의 정보를 user 구조에서 추출해서 저장한다.

```
%   1. 로그인 유저의 선호 정보 및 전체 유저 정보 구조화

% 로그인 유저 아이디 불러오기
filename = '..\data\database\user_data.txt';
if exist(filename, 'file') % user_data.txt 파일 존재 여부 확인
    fid = fopen(filename, 'rt'); % 읽기 모드로 열기
    id = strtrim(fgetl(fid)); % 첫 줄에서 학번 읽기
    fclose(fid);
else
    errordlg('user_data.txt 파일이 없습니다. 먼저 로그인하세요.', '오류');
    return;
end


% 유저를 정보 파일을 불러오기
filename1 = '..\data\database\user_want.txt';
filename2 = '..\data\database\user_prefer.txt';
filename3 = '..\data\database\user.txt';
filename4 = '..\data\database\user_timetable.txt';
filename5 = '..\data\database\matched_user.txt';

% 유저를 정보 파일 read 모드로 열기
fid1 = fopen(filename1, 'rt');
fid2 = fopen(filename2, 'rt');
fid3 = fopen(filename3, 'rt');
fid4 = fopen(filename4, 'rt');
fid5 = fopen(filename5, 'wt');
```

그림 75 로그인 한 유저 정보 저장

```
% 선호 정보의 총 데이터 수가 같으므로 한번에 구조에 저장 (want, prefer 한번에 저장)
% 유저들의 선호 정보를 담을 빈 struct 구조 생성
user_pre = {};
i = 1; % 인덱스 용 숫자
while ~feof(fid1)   % 파일의 끝에 도달하지 않았으면 반복
    line1 = fgetl(fid1);   % 해당 열 문자를 받기
    line2 = fgetl(fid2);
    if line1 == -1   % 마지막 열일 경우 반복문 종료
        break
    end
    linedata1 = strsplit(line1,'\t');
    linedata2 = strsplit(line2,'\t');   % 유저들의 정보를 tap 을 기준으로 정보를 쪼개고 user struct에 저장

    user_pre(i).id = linedata1{1};
    user_pre(i).want = linedata1{2};

    user_pre(i).gender = linedata2{2};
    user_pre(i).major = linedata2{3};
    user_pre(i).grade = linedata2{4};
    user_pre(i).day = linedata2{5};


    i = i+1;  % 다음 user struct에 저장하기 위해 index 업데이트
end

fclose(fid1);
fclose(fid2);
```

그림 76 유저들의 선호 정보 저장

```
% 로그인 유저의 정보 인덱스 저장 (유저 선호 정보 중)
preidx = find(strcmp({user_pre.id},id));
user_mypre = user_pre(preidx);
user_pre(preidx) = [];

% 전체 유저들의 정보 저장
% 유저들의 정보를 담을 빈 struct 구조 생성
user = {};
i = 1; % 인덱스용 숫자
while ~feof(fid3)   % 파일의 끝에 도달하지 않았으면 반복
    line = fgetl(fid3);   % 해당 열 문자를 받기

    if line == -1   % 마지막 열일 경우 반복문 종료
        break
    end

    linedata = strsplit(line,'\t');   % 유저의 정보를 tap 을 기준으로 정보를 쪼개고 user struct에 저장

    user(i).id = linedata{1};
    user(i).major = linedata{3};
    user(i).gender = linedata{4};
    user(i).grade = linedata{5};
    user(i).kakao = linedata{6};

    i = i+1;  % 다음 user struct에 저장하기 위해 index 업데이트
end

% 나와 매칭 방지
% 로그인 유저의 정보 인덱스 저장, 전체 유저 정보 중 해당 정보 삭제 (전체 유저 정보 중)
useridx = find(strcmp({user.id},id));
mine = user(useridx);
user(useridx) = [];

fclose(fid3);
```

그림 77 로그인 유저의 선호 저장, 전체 유저 데이터 구조화

```
% 선호 정보가 있는 유저의 id 정보 cell로 저장
j = 1;
usernew = {};
for i = 1:length(user)
    if strcmp({user_pre(j).id},user(i).id)
        usernew(j) = {user(i)};
        j = j+1;
        if j == length(user_pre)+1
            break
        end
    end
end
```

그림 78 선호 정보가 있는 유저 구조화

이후 usernew 셀에서 첫 번째 유저의 데이터부터 로그인 한 유저의 선호 정보와 탐색하고자 하는 유저의 선호 정보를 비교한다. 두 명의 유저의 선호 정보가 동일한 경우, 선호하는 만남의 종류에 따라 시간표 정보를 비교한다. 이때 모든 조건에 부합하는 유저들의 만날 수 있는 요일, 시간, 카카오톡 아이디, 전공, 성별, 학년 정보를 matched 구조에 day, time, kakao, major, gender, grade 필드로 각각 저장한다.

유저의 선호 정보는 다음과 같이 선택할 수 있다. :

- 만남의 종류: 밥 친구, 공부, 친구,
- 성별: 남자, 여자, all,
- 학과: same, other, all,
- 학년: 1, 2, 3, 4, 5, all,
- 요일: mon, tue, wed, thu, fri.

먼저 전체 유저 정보 (usernew)에 저장된 순서에 따라 탐색하는 유저의 선호 만남 종류와 로그인 한 유저의 선호 만남이 같은 경우 나머지 4개의 선호 정보를 비교한다. 이때 탐색하는 유저가 선호하는 요일은 만남 성사의 가능성을 높이기 위해 경우에서 배제하였다. 각 경우의 수는 아래와 같다. 이때 '동일'은 선택한 선호 정보가 일치하는 경우를 의미한다.

- 성별 무관, 전공 무관, 학년 무관,
- 성별 무관, 전공 무관, 학년 동일,
- 성별 무관, 학년 무관, 전공 동일,
- 성별 무관, 학년 동일, 전공 동일,
- 전공 무관, 학년 무관, 성별 동일,
- 전공 무관, 학년 동일, 성별 동일,
- 학년 무관, 성별 동일, 전공 동일,
- 학년 동일, 성별 동일, 전공 동일.

로그인 한 유저의 선호 정보에 따라 위의 경우를 조건문으로 나누어 확인한다. 각 조건에 추가로 로그인 한 유저의 회원 정보가 탐색하는 유저의 선호 정보에 부합하는 경우, matcheduser 구조에 id, gender, major, grade, kakao 총 5개의 필드로 정보를 저장한다. 결과적으로, matcheduser 구조에는 '선호 조건이 서로 부합하는 유저'의 정보가 저장된다.

```
% 로그인 유저의 선호와 성별, 전공, 학년이 같으면 해당 유저 정보 저장
for i = 1:length(usernew)


    matchinguser = usernew{i} ;
    idindex = find(strcmp({user_pre.id},matchinguser.id));
    idindexx = find(strcmp({user.id},matchinguser.id));

    %==================================================================
    % 내 만남 선호와 상대의 만남 선호가 같은 경우
    if strcmp(user_mypre.want, user_pre(idindex).want)

        % 내 선호가 성별 무관 + 전공 무관 + 학년 무관인 경우
        % 상대의 선호가 무관이거나 내 정보와 동일한 경우
        if strcmp(user_mypre.gender,'all') && ...
           strcmp(user_mypre.major, 'all') && ...
           strcmp(user_mypre.grade, 'all') && ...
           (strcmp(user_pre(idindex).gender, 'all') || strcmp(user_pre(idindex).gender,mine.gender)) && ...
           (strcmp(user_pre(idindex).major, 'all') || ... % 상대의 전공 선호 무관
           strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 상대의 전공 선호 동일
           strcmp(user_pre(idindex).major, 'other') && ~strcmp(user(idindexx).major,mine.major)) && ... % 상대의 전공 선호 비동일
           (strcmp(user_pre(idindex).grade, 'all') || strcmp(user_pre(idindex).grade,mine.grade))
            matcheduser(c).id = matchinguser.id;
            matcheduser(c).gender = matchinguser.gender;
            matcheduser(c).major = matchinguser.major;
            matcheduser(c).grade = matchinguser.grade;
            matcheduser(c).kakao = matchinguser.kakao;
            c=c+1;

            % 내 선호가 성별 무관인 경우
            % 상대의 성별 선호가 성별 무관이거나 내 정보와 동일한 경우
        elseif strcmp(user_mypre.gender, 'all') && ...
               (strcmp(user_pre(idindex).gender, 'all') || strcmp(user_pre(idindex).gender,mine.gender))

            % 내 선호가 성별 무관 + 전공 무관 + 학년 동일인 경우
            % 상대의 선호가 전공 무관이거나 내 정보와 동일한 경우
            if strcmp(user_mypre.major,'all') && ...
                   (strcmp(user_pre(idindex).major, 'all') || ...  % 상대의 전공 선호 무관
                   strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx),mine.major) ||... % 상대의 전공 선호 동일
                   strcmp(user_pre(idindex).major,'other') && ~strcmp(user(idindexx),mine.major)) % 상대의 전공 선호 비동일
               % 선호 학년과 상대 학년이 같은 경우
               if strcmp(user_mypre.grade, matchinguser.grade)
                   matcheduser(c).id = matchinguser.id;
                   matcheduser(c).gender = matchinguser.gender;
                   matcheduser(c).major = matchinguser.major;
                   matcheduser(c).grade = matchinguser.grade;
                   matcheduser(c).kakao = matchinguser.kakao;
                   c=c+1;
               end

               % 내 선호가 성별 무관 + 학년 무관 + 전공 동일인 경우
               % 상대의 선호가 학년 무관이거나 내 정보와 동일한 경우
            elseif  strcmp(user_mypre.grade,'all') && ...
                   (strcmp(user_pre(idindex).grade, 'all') || strcmp(user_pre(idindex).grade,mine.grade))
               % 선호 전공과 상대 전공이 같은 경우
               if (strcmp(user_mypre.major,'same') && strcmp(user(idindexx),mine.major) ||... % 나의 전공 선호 동일
                   strcmp(user_mypre.major,'other') && ~strcmp(user(idindexx),mine.major)) && ...
                   (strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 상대의 전공 선호 동일
                   strcmp(user_pre(idindex).major,'other') && ~strcmp(user(idindexx).major,mine.major))
                   matcheduser(c).id = matchinguser.id;
                   matcheduser(c).gender = matchinguser.gender;
                   matcheduser(c).major = matchinguser.major;
                   matcheduser(c).grade = matchinguser.grade;
                   matcheduser(c).kakao = matchinguser.kakao;
                   c=c+1;
               end

               % 내 선호가 성별 무관 + 학년 동일 + 전공 동일인 경우
            else
               % 내 학년, 전공 선호와 상대 학년, 전공이 같을 경우
               if (strcmp(user_mypre.major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 나의 전공 선호 동일
                   strcmp(user_mypre.major,'other') && ~strcmp(user(idindexx).major,mine.major)) && ...
                   (strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 상대의 전공 선호 동일
                   strcmp(user_pre(idindex).major,'other') && ~strcmp(user(idindexx).major,mine.major)) && ...
                   strcmp(user_mypre.grade, matchinguser.grade)
                   matcheduser(c).id = matchinguser.id;
                   matcheduser(c).gender = matchinguser.gender;
                   matcheduser(c).major = matchinguser.major;
                   matcheduser(c).grade = matchinguser.grade;
                   matcheduser(c).kakao = matchinguser.kakao;
                   c=c+1;
               end
            end
```

그림 79 조건에 맞는 회원 탐색 1

```
        % 내 선호가 전공 무관인 경우
        % 상대의 전공 선호가 전공 무관이거나 내 전공과 동일한 경우
    elseif strcmp(user_mypre.major, 'all') && ...
           (strcmp(user_pre(idindex).major,  'all') || ...  % 상대의 전공 선호 무관
           strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 상대의 전공 선호 동일
           strcmp(user_pre(idindex).major,'other') && ~strcmp(user(idindexx).major,mine.major)) % 상대의 전공 선호 비동일

        % 내 선호가 전공 무관 + 성별 무관인 경우 + 학년 동일인 경우 위에서 카운트 완료
        % 내 선호가 전공 무관 + 학년 무관 + 성별 동일인 경우
        % 상대의 학년 선호가 무관이거나 내 정보와 동일한 경우
        if  strcmp(user_mypre.grade,'all') && ...
               (strcmp(user_pre(idindex).grade, 'all') || strcmp(user_pre(idindex).grade,mine.grade))
            % 내 성별 선호와 상대 성별이 같은 경우
            if strcmp(user_mypre.gender, matchinguser.gender)
                matcheduser(c).id = matchinguser.id;
                matcheduser(c).gender = matchinguser.gender;
                matcheduser(c).major = matchinguser.major;
                matcheduser(c).grade = matchinguser.grade;
                matcheduser(c).kakao = matchinguser.kakao;
                c=c+1;
            end

            % 내 선호가 전공 무관 + 성별 동일 + 학년 동일인 경우
        else
            if  strcmp(user_mypre.gender, matchinguser.gender) && ...
                   strcmp(user_mypre.grade, matchinguser.grade)
                matcheduser(c).id = matchinguser.id;
                matcheduser(c).gender = matchinguser.gender;
                matcheduser(c).major = matchinguser.major;
                matcheduser(c).grade = matchinguser.grade;
                matcheduser(c).kakao = matchinguser.kakao;
                c=c+1;
            end
        end

        % 내 선호가 학년 무관인 경우
        % 상대의 학년 선호가 학년 무관이거나 내 학년과 동일한 경우
    elseif strcmp(user_mypre.grade, 'all') && ...
           (strcmp(user_pre(idindex).grade,  'all') || strcmp(user(idindexx).grade,mine.grade))

        % 내 선호가 학년 무관 + 전공 무관 + 성별 동일인 경우 위에서 카운트 완료
        % 내 선호가 학년 무관 + 성별 무관 + 전공 동일인 경우 위에서 카운트 완료
        % 내 선호가 학년 무관 + 성별 동일 + 전공 동일인 경우
        if  strcmp(user_mypre.gender, matchinguser.gender) && ...
            (strcmp(user_mypre.major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 나의 전공 선호 동일
             strcmp(user_mypre.major,'other') && ~strcmp(user(idindexx).major,mine.major)) && ...
            (strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 상대의 전공 선호 동일
             strcmp(user_pre(idindex).major,'other') && ~strcmp(user(idindexx).major,mine.major))
            matcheduser(c).id = matchinguser.id;
            matcheduser(c).gender = matchinguser.gender;
            matcheduser(c).major = matchinguser.major;
            matcheduser(c).grade = matchinguser.grade;
            matcheduser(c).kakao = matchinguser.kakao;
            c=c+1;
        end
        % 내 선호가 성별 동일 + 전공 동일 + 학년 동일인 경우
    else
        if strcmp(user_mypre.gender, matchinguser.gender) && ...
           (strcmp(user_mypre.major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 나의 전공 선호 동일
            strcmp(user_mypre.major,'other') && ~strcmp(user(idindexx).major,mine.major)) && ...
           (strcmp(user_pre(idindex).major,'same') && strcmp(user(idindexx).major,mine.major) ||... % 상대의 전공 선호 동일
            strcmp(user_pre(idindex).major,'other') && ~strcmp(user(idindexx).major,mine.major)) && ...
            strcmp(user_mypre.grade, matchinguser.grade)
            matcheduser(c).id = matchinguser.id;
            matcheduser(c).gender = matchinguser.gender;
            matcheduser(c).major = matchinguser.major;
            matcheduser(c).grade = matchinguser.grade;
            matcheduser(c).kakao = matchinguser.kakao;

            c = c+1;
        end
    end
```

그림 80 조건에 맞는 회원 탐색 2

조건에 맞는 유저가 없는 경우 다시 만남의 종류를 선택하는 창으로 돌아가 선호를 재설정한다.

```
if length(matcheduser) == 0
    h = msgbox('조건에 맞는 사람이 없습니다.');
    uiwait(h);
    run mainhome.m;
    close(matching_gui);
else
```

그림 81 조건에 맞는 유저가 없는 경우

마지막으로 로그인 한 유저가 선호하는 만남 종류에 따라 조건이 맞는 유저들과의 시간표를 비교하여 최종적으로 만남이 성립될 수 있는 유저를 판별한다. user_timetable.txt에서 전체 유저의 시간표를 user_timetable 구조에 저장한다. 이때 로그인 한 유저가 선호하는 요일 시간표에 대해 로그인 한 유저의 시간표 데이터를 user_day 셀에, matcheduser 구조에 기반하여 탐색하려는 유저의 시간표 데이터를 pre_day 셀에 저장한다. 또한 각 유저들의 회원 정보 (kakao 아이디, 전공, 성별, 학년)를 함께 저장하였다. pre_day 셀에는 matcheduser 구조에서 선호 요일 시간표 데이터가 있는 유저들의 시간표 데이터와 회원 정보가 저장된다.

```
classname = ["충무관", "다산관", "학술정보원", "영실관", "우정당", "박물관", "세날관", "용덕관", ...
             "진관홀", "광개토관", "세종관", "군자관", "학생회관", "정현관", "대양홀", "이당관", "애지헌"];

% 유저들의 시간표 데이터 구조화
% 시간표 데이터 저장할 빈 struct 구조 생성
user_timetable = {} ;
% 시간표 데이터 그대로 저장할 배열 생성
i = 1; % 인덱스 용 숫자
while ~feof(fid4)   % 파일의 끝에 도달하지 않았으면 반복
    line4 = fgetl(fid4);   % 해당 열 문자를 받기

    if line4 == -1   % 마지막 열일 경우 반복문 종료
        break
    end

    linedata4 = strsplit(line4,'\t') ;  % 유저들의 정보를 tap 을 기준으로 정보를 쪼개고 user struct에 저장
    user_timetable(i).id = linedata4{1};
    user_timetable(i).day = linedata4{2};
    user_timetable(i).class1 = linedata4{3};
    user_timetable(i).class2 = linedata4{4};
    user_timetable(i).class3 = linedata4{5};
    user_timetable(i).class4 = linedata4{6};
    user_timetable(i).class5 = linedata4{7};
    user_timetable(i).class6 = linedata4{8};
    user_timetable(i).class7 = linedata4{9};

    i = i+1;  % 다음 user struct에 저장하기 위해 index 업데이트
end

fclose(fid4);
```

그림 82 시간표 구조화

```
% timetable에 따른 매칭
% pre_day 는 선호 유저들의 시간표가 선호 요일과 같은 경우 시간표 정보 받기
pre_day = {} ;
c = 1;
for i = 1:length(user_timetable)
    for j = 1:length(matcheduser)
        if strcmp(matcheduser{j}.id , user_timetable(i).id) && ...
                strcmp(user_mypre.day , user_timetable(i).day)
            pre_day{c,1} = user_timetable(i).class1;
            pre_day{c,2} = user_timetable(i).class2;
            pre_day{c,3} = user_timetable(i).class3;
            pre_day{c,4} = user_timetable(i).class4;
            pre_day{c,5} = user_timetable(i).class5;
            pre_day{c,6} = user_timetable(i).class6;
            pre_day{c,7} = user_timetable(i).class7;

            kakaoidx = find(strcmp({user.id},matcheduser{j}.id));
            pre_day{c,8} = user(kakaoidx).kakao;
            pre_day{c,9} = user(kakaoidx).major;
            pre_day{c,10} = user(kakaoidx).gender;
            pre_day{c,11} = user(kakaoidx).grade;

            c = c+1;
        end
    end
end

% user_day 는 로그인 유저의 요일에 맞는 시간표 받기
user_day = {} ;
for i = 1:length(user_timetable)
    if strcmp(id, user_timetable(i).id) && ...
            strcmp(user_mypre.day , user_timetable(i).day)
        user_day{1,1} = user_timetable(i).class1;
        user_day{1,2} = user_timetable(i).class2;
        user_day{1,3} = user_timetable(i).class3;
        user_day{1,4} = user_timetable(i).class4;
        user_day{1,5} = user_timetable(i).class5;
        user_day{1,6} = user_timetable(i).class6;
        user_day{1,7} = user_timetable(i).class7;
    end
end

for p = 1:length(pre_day(:,1))
    for t = 1:7
        pre_day{p,t} = str2double(pre_day{p,t});
        if p == 1
            user_day{1,t} = str2double(user_day{1,t});
        end
    end
end
```

그림 83 조건에 맞는 유저들의 시간표 구조화

각 만남 종류에 따라 matched 구조에 해당 요일, 탐색하는 유저들의 kakao 아이디, 만날 수 있는 시간, 해당 수업이 끝나는 건물, 전공, 성별, 학년 정보가 저장된다. 점심 식사 친구를 만들고 싶은 유저의 경우에는 2교시 끝나는 건물이 같고, 3교시에 수업이 없거나 3교시 끝나는 건물이 같고, 4교시에 수업이 없는 경우 매칭된다. 공부 친구의 경우, 시간표 상 마지막 수업이 끝나는 건물이 같을 경우 매칭된다. 특정 교시의 건물이 같고, 다음 교시부터 마지막 교시까지 수업이 없거나, 학술정보원에 위치한 경우 매칭된다. 그냥 친구의 경우에는, 각 수업이 같은 건물에서 이루어지는 경우 매칭된다.

```
% 선호하는 만남에 따라 매칭
% 매칭되는 사람 정보 저장할 배열
matched = {};
a = 1; % 중복되어 매칭되는 경우를 위한 인덱스
for p = 1:length(pre_day(:,1))
    t = 1;
    switch user_mypre.want
        % 점심 식사 친구
        case "meal"
            % 2교시 끝나는 건물이 같고 3교시 수업이 없는 경우 (12시 식사)
            if pre_day{p,2} == user_day{2} && pre_day{p,3} == 0 && user_day{3} == 0
                matched(a).day = user_pre.day;
                matched(a).kakao = pre_day{p,8};
                matched(a).time = 2; % class 2 이후 점심
                matched(a).major = pre_day{p,9};
                matched(a).gender = pre_day{p,10};
                matched(a).grade = pre_day{p,11};
                matched(a).place = classname(user_day{2});
                % 3교시 끝나는 건물이 같고 4교시 수업이 없는 경우 (13시 30분 식사)
            elseif pre_day{p,3} == user_day{3} && pre_day{p,4} == 0 && user_day{4} == 0
                matched(a).day = user_pre.day;
                matched(a).kakao = pre_day{p,8};
                matched(a).time = 3; % class 3 이후 점심
                matched(a).major = pre_day{p,9};
                matched(a).gender = pre_day{p,10};
                matched(a).grade = pre_day{p,11};
                matched(a).place = classname(user_day{3});
            end
            a = a + 1;
```

그림 84 밥 친구 매칭

```
elseif strcmp(user_mypre.want,'study') %끝나는 수업과 건물이 같은경우
    a=1;
    for p=1:length(pre_day(:,1))
        user_last_class=[];
        pre_last_class=[];
        for i=7:-1:1
            if user_day{i}~=0
                user_last_class=[i user_day{i}];
                idxxxx = i;
                break;
            end
        end
        for i=7:-1:1
            if pre_day{p,i}~=0
                pre_last_class=[i pre_day{p,i}];
                idxxxx = i;
                break;
            end
        end
        if user_last_class(1)==pre_last_class(1) && user_last_class(2)==pre_last_class(2)
            matched(a).day = user_mypre.day;
            matched(a).kakao = pre_day{p,8};
            matched(a).time = idxxxx ;
            matched(a).major = pre_day{p,9};
            matched(a).gender = pre_day{p,10};
            matched(a).grade = pre_day{p,11};
            matched_places(a) = char(classname(user_last_class(2)));

            a = a + 1;
        end
```

그림 85 공부 친구 매칭

```
elseif strcmp(user_mypre.want,'friend')
    for p=1:length(pre_day(:,1))
        c=0;
        times = [];
        places = [];
        for t = 1:7
            if pre_day{p,t} == user_day{t} && ...
                    (pre_day{p,t} ~= 0 && user_day{t} ~= 0) && ...
                    (pre_day{p,t} ~= 3 && user_day{t} ~= 3)
                c=c+1;
                times=[times t];
            end

        end
        time(p).time=times;
        if c>0
            class_count=[class_count; p, c;];
        end
    end

    for i=1:length(class_count(:,1))
        for j= 1:length(class_count(:,1))
            if class_count(i,2)>class_count(j,2)
                temp=class_count(i,:);
                class_count(i,:)=class_count(j,:);
                class_count(j,:)=temp;
            end
        end
    end
    matched_places={};
    for i=1:length(class_count(:,1))
        p=class_count(i,1);
        times=time(p).time;
        for j=1:length(times)
            matched_places{i,j}=char(classname(user_day{times(j)}))
        end
        matched(i).day = user_mypre.day;
        matched(i).kakao = pre_day{p,8};
        matched(i).time = time(p).time; % class t 이후 공부
        matched(i).major = pre_day{p,9};
        matched(i).gender = pre_day{p,10};
        matched(i).grade = pre_day{p,11};

    end
end
```

그림 86 친구 매칭

모든 경우의 수를 따져 선호 조건과 시간대가 일치하는 경우, 데이터베이스 폴더 내의 matched_user.txt 파일에 matched 구조에 담겨있는 모든 조건에 부합하는 유저의 정보가 저장된다. matched 구조에 부합하는 유저의 정보가 저장되면 매칭에 성공한 것이고, 매칭 결과를 볼 수 있도록 해당 파일이 실행되며, 매칭에 실패 한 경우 '조건에 맞는 사람을 찾을 수 없습니다.' 라는 알림 창과 함께 다시 선호하는 만남 종류를 선택하는 창으로 돌아가 다시금 선호 조건을 설정하게 된다.

```
if ~isempty(matched) % matched 구조체가 비어 있지 않은 경우
    fprintf(fid5, '%s\t%s\t%s\t%s\t%s\t%s\t%s\n','Want','Day','Major','Gender','Grade','KAKAO ID','When','Where');
    for i = 1:length(matched)
    fprintf(fid5, '%s\t%s\t%s\t%s\t%s\t%s\t', user_mypre.want,matched(i).day,matched(i).major,matched(i).gender,...
        matched(i).grade,matched(i).kakao);
        for j = 1:length(matched(i).time)
            if j == length(matched(i).time)
                fprintf(fid5,'%d교시(%s)\n',matched(i).time(j),matched(i).place(j));
            else
                fprintf(fid5,'%d교시(%s), ',matched(i).time(j),matched(i).place(j));
            end
        end
    end
    fclose(fid5);
    run result.m;
else
    fclose(fid5);
    h = msgbox('조건에 맞는 사람을 찾을 수 없습니다.');
    uiwait(h);
    run mainhome.m;
end
```

그림 87 매칭된 유저 정보 저장

matched_user.txt

Want	Day	Major	Gender	Grade	KAKAO ID	When	Where
friend	mon	물리천문학과	남자	2	0009	3교시(군자관)	
friend	mon	수학통계학과	남자	2	0010	4교시(군자관),	5교시(군자관)
friend	mon	물리천문학과	여자	2	0011	4교시(군자관)	

그림 88 매칭된 유저 정보 저장 결과 예시

2.3.9 결과

결과 GUI 창에서 코드는 매칭된 결과를 확인하고, 매칭된 회원들의 정보를 화면에 표시하는 결과 GUI (result.fig) 창을 구현한다. 결과 창은 매칭된 인원의 수와 매칭된 회원 중 한 명의 정보를 보여준다. 사용자는 '매칭 수락' 버튼을 통해 정보를 확인할 수 있다. 결과 창에서는 매칭된 사용자 정보가 저장된 matched_user.txt 파일을 읽는다. 해당 파일이 존재하지 않거나 열 수 없는 경우, 오류 메시지를 출력하고 작업을 중단한다. 파일이 정상적으로 열리면 fgetl 을 사용해 데이터를 한 줄씩 읽어 file_content 리스트에 저장한다. 이 리스트는 이후 매칭된 사용자 데이터를 처리하는 데 이용된다. 파일의 첫 번째 줄은 데이터 필드명을 나타내는 헤더로 이용되며, 이후 줄에는 매칭된 사용자 정보가 저장된다. 선택된 줄의 데이터를 tab (\t) 으로 나눠 각 필드 (예: 매칭 목적, 요일, Kakao ID 등)로 분리한 후, 사용자에게 읽기 쉬운 형태로 생성한다. 매칭된 사용자 정보를 바탕으로 구축한 문자열을 GUI의 텍스트 컴포넌트 (handles.text_display)에 표시한다. 혹 파일에 매칭 데이터가 없는 경우, '텍스트 파일에 데이터가 없습니다' 라는 메시지를 출력한다. 파일을 열 수 없는 경우 오류 메시지를 출력하고 실행을 중단한다.

```
function pushbutton1_Callback(hObject, eventdata, handles)
filename = 'matched_user.txt';
fid = fopen(filename, 'rt');
handles.n=handles.n+1;
if handles.n>handles.lineCount
    h=msgbox('모든 친구를 찾았습니다!');
    uiwait(h);
    handles.n=handles.lineCount;
end
for i=1:handles.n
    line=fgetl(fid);
end
fclose(fid);
data = strsplit(line, '\t');
data_n=length(data);
handles.kt=data{6};
if length(data)>7
    str=sprintf('매칭된 인원 수: %d\n목적: %s\n요일: %s\nClass: ',handles.lineCount, data{1}, data{2});
    for i=7:data_n
        str=sprintf('%s%s',str,data{i});
    end
    formatted_str=sprintf('%s\n학과: %s\n성별: %s\n학년: %s\nKakaoID: %s\n',str,data{3},data{4}, data{5}, data{6});
else
    formatted_str = sprintf('매칭된 인원 수: %d\n목적: %s\n요일: %s\nClass: %s\n학과: %s\n성별: %s\n학년: %s\nKakaoID: %s\n', ...
        handles.lineCount-1, data{1}, data{2}, data{7}, data{3}, ...
        data{4}, data{5}, data{6});
end

% 6. 텍스트 컴포넌트에 표시
set(handles.text_display, 'String', formatted_str);
guidata(hObject,handles);
end
```

그림 89 매칭 수락 콜백 함수

매칭 수락 버튼 콜백 함수는 먼저 matched_user.txt 파일에 저장된 매칭된 회원들을 순서대로 보여준다. 파일에서 매칭된 사용자 데이터를 읽어 리스트로 저장한 후, 사용자가 선택된 매칭 결과를 확인할 수 있도록 GUI 화면에 결과를 보여준다. 매칭 수락 버튼을 계속 누를 때마다 해당 파일의 회원들이 순서대로 나타나며, 마지막 회원 정보까지 보여주었다면, '모든 친구를 찾았습니다!' 안내메시지 창이 뜬다.

2.4 프로그램 사용 방법

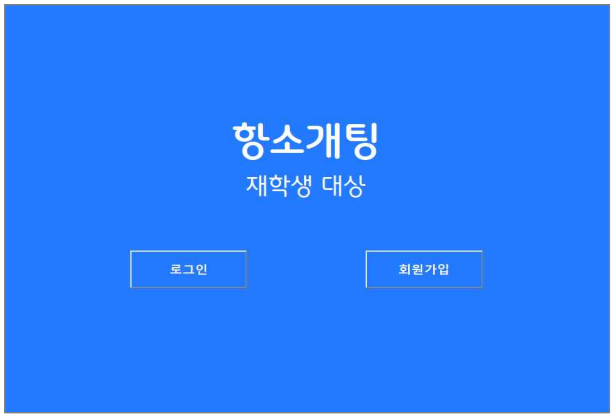


그림 90 example.fig

1. example.m을 실행시킨다. 이미 가입되어 있는 경우, 로그인 버튼을, 그렇지 않을 경우, 회원가입 버튼을 누른다. (로그인 버튼을 누를 경우 바로 3단계로 넘어간다.)

그림 91 register.fig

2. 각 정보를 입력한 뒤 완료 버튼을 누른다.

그림 92 login.fig

3. 회원가입을 한 학번과 비밀번호를 입력 후, 로그인 버튼을 누른다.

그림 93 timetable.fig

4. 시간표 업로드를 먼저 눌러, csv 파일을 업로드 한다. 시간표 업데이트 버튼을 눌러 화면에 표시되는 자신의 수업을 확인하고 시작하기 버튼을 누른다.

그림 94 mainhome.fig

그림 95 modify.fig

5. 설정 버튼을 눌러, 자신의 회원 정보를 수정할 수 있다

(1) 3개의 버튼 중 하나를 눌러, 자신이 원하는 친구의 유형을 선택할 수 있다.

(2) 자신의 회원 정보를 수정한 뒤, 수정 완료 버튼을 누르면 이전 화면으로 돌아간다.

그림 96 setting.fig

6. 자신이 선호하는 성별, 학과, 학년, 요일을 선택하고, 시작 버튼을 누른다.

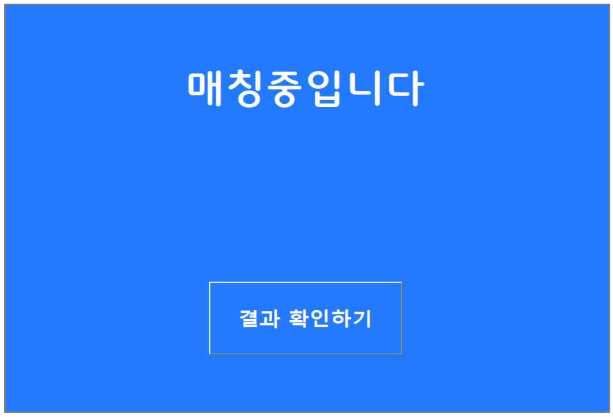


그림 98 msgbox

그림 97 matching_gui.fig

7. 결과 확인하기 버튼을 눌러, 결과를 확인한다. 조건에 맞는 사람이 존재하지 않는 경우 그림46 알림창이 뜨고, 알림창을 제거할 때, mainhome.fig 로 돌아간다. 조건에 맞는 사람이 존재할 때, 다음 단계 (그림 99)로 넘어간다.

그림 99 result.fig (before)

그림 100 result.fig (after)

8. 매칭 수락 버튼을 누르면, 조건에 맞는 사람 중에서 무작위로 한 명의 정보를 알려준다. 이 정보를 이용해 매칭된 사람에게 연락한다.

2.5 프로그램 검증

2.5.1 검증 데이터베이스 설정

프로그램 검증에 앞서 user, user_timetable, user_want, user_prefer 데이터들을 다음과 같이 설정했다.

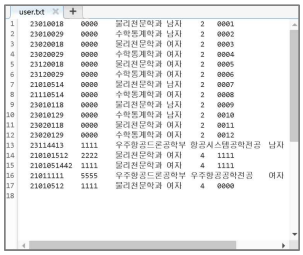


그림 68 user.txt

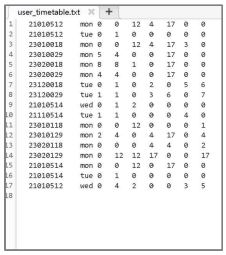


그림 69 user_timetable.txt

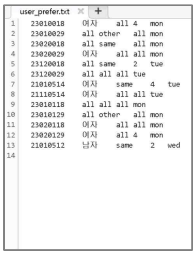


그림 103 user_prefer.txt

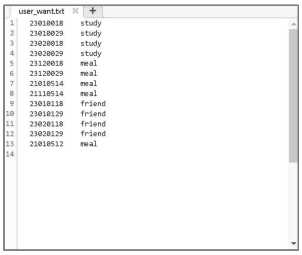


그림 104 user_want.txt

2.5.2 각 단계 별 뜨는 안내메시지 창

회원가입 창에서 입력 정보가 빠졌을 때, 입력 정보의 저장에 성공했을 때, 입력 정보가 이미 저장되어 있을 때 메시지 창을 아래 사진과 같이 확인하였다.

그림 105 입력 정보 누락

그림 106 입력 정보 저장

그림 107 이미 저장된 경우

로그인 창에서 로그인 성공, 실패의 경우에 따른 메시지 창은 아래와 같다.

그림 108 회원가입 정보와 일치

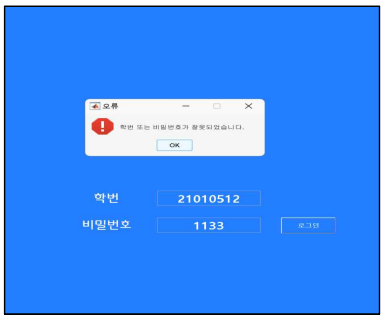


그림 109 회원가입 정보와 불일치

정보 수정 (설정) 창에서 입력 정보가 온전한 경우는 다음 인터페이스로 넘어가 저장되며, 입력 정보가 부족할 시 아래와 같은 메시지 창이 뜬다.

그림 110 입력 정보 누락

2.5.3 매칭 결과 검증

2.5.3.a 타겟팅 검증

2.5.1에서의 회원 데이터들을 기반으로 특정 타겟을 지정하여 몇 가지의 목적과 결과를 확인하고자 했다. 사용자는 21010512로 통합하였다. 21010512 사용자의 시간표 정보는 다음과 같다.:

- 월요일 3교시 군자관, 4교시 영실관, 5교시 애지헌,
- 화요일 2교시 충무관,
- 수요일 2교시 영실관, 3교시 다산관, 6교시 학술정보원, 7교시 우정당.

그림 111 밥
친구/남자/same/2/tue
매칭 결과

사용자의 선호가 밥 친구, 남자, same, 2, tue 일 때 그림 111과 같이 2교시 직후 밥 친구가 매칭되었다. 21010514 (0007) 결과를 확인하였다.

그림 112 밥
친구/남자/same/2/wed
매칭 결과

사용자의 선호가 밥 친구, 남자, same, 2, wed 일 때 그림 112와 같이 3교시 직후 밥 친구가 매칭되었다. 21010514 (0007) 결과를 확인하였다.

그림 113
공부/남자/other/2/mon
매칭 결과

사용자의 선호가 공부, 남자, other, 2, mon 일 때 그림 113과 같이 5교시 직후 공부할 친구가 매칭되었다. 23010029(0002) 결과를 확인할 수 있다.

그림 114
공부/여자/same/all/mon
매칭 결과

사용자의 선호가 공부, 여자, same, all, mon 일 때 그림 114와 같이 5교시 직후 공부할 친구가 매칭되었다. 23020018(0003) 결과를 확인할 수 있다.

그림 115
친구/all/other/2/mon
매칭 결과

사용자의 선호가 친구, all, other, 2, mon 일 때 그림 115와 같이 4,5 교시에 같이 수업을 듣는 친구가 매칭되었다. 23010129(0010) 결과를 확인할 수 있다.

그림 116 친구/all/all/all/mon
매칭 결과

그림 117 친구/all/all/all/mon
매칭 결과_2

그림 118 친구/all/alla/ll/mon
매칭 결과_3

사용자의 선호가 친구, all, all, all, mon 일 때 그림 116과 같이 수업이 같은 여러 친구가 매칭되었다. '매칭 수락' 버튼을 누르면 다른 매칭 결과도 볼 수 있다.

그림 119 밥 친구/남자
/same/3/mon 매칭 결과

사용자의 선호가 밥 친구, 남자, same, 3, mon 일 때 그림 119과 같이 '조건에 맞는 사람이 없습니다' 메시지가 나타난다.

2.5.3.b 랜덤 회원 대상 검증

매칭 알고리즘을 견고히 검증하기 위해서 더 많은 회원 데이터 정보를 얻기 위해서 랜덤 회원을 임의로 만드는 함수를 작성하였다.

*randomuser 스크립트

이 코드에서는 랜덤 회원의 학번과, 시간표 정보, want, 학년, 성별을 randi 함수를 통해 무작위로 생성하고 이를 user.txt , user_prefer.txt, user_want.tx, user_timetable.txt 에 저장한다. 이 코드를 사용하여 랜덤회원 2000명을 만들었고 이를 통해 매칭 알고리즘 검증을 실시했다.

사용자는 21011324로, 시간표 정보는 다음과 같다.:

- 월요일 5교시 군자관, 6교시 광개토관,
- 화요일, 목요일 2, 3, 5교시 충무관.

각 목적에 따라 몇 명과 매칭되는지를 확인하였고, 결과는 다음과 같다.

그림 120 밥 친구/all/all/all/tue 매칭 결과

그림 121 공부/all/all/all/tue 매칭 결과

그림 122 친구/all/all/all/tue 매칭 결과

각 1명, 0명, 25명과 매칭되었고 이를 매칭 회원 수와 학번을 반환하는 확인 알고리즘 코드를 통해 비교하였다.

두 번째로 검증한 사용자는 21010521로, 시간표 정보는 다음과 같다.:

- 화요일 3교시 군자관, 4교시 영실관, 5교시 애지헌.

각 목적에 따라 몇 명과 매칭되는지를 확인하였고, 결과는 다음과 같다.

그림 123 밥 친구/all/all/all/tue 매칭 결과

그림 124 공부/all/all/all/tue 매칭 결과

그림 125 친구/all/all/all/tue 매칭 결과

각 0명, 0명, 29명과 매칭되었고 이를 매칭 회원 수와 학번을 반환하는 확인 알고리즘 코드를 통해 비교하였다.

세 번째로 검증한 사용자는, 랜덤으로 생성한 2000명의 회원 중 한명인 20707784로,

시간표 정보는 다음과 같다.:

- 화요일 이당관-광개토관-대양홀-새날관-광개토관-충무관-이당관

그림 126 밥 친구/all/all/all/tue 매칭 결과

그림 127 공부/all/all/all/tue 매칭 결과

그림 128 친구/all/all/all/tue 매칭 결과

각 0명, 12명, 60명과 매칭되었고 이를 매칭 회원 수와 학번을 반환하는 확인 알고리즘 코드를 통해 비교하였다.

2.6 결과 및 분석

- meal 매칭

: meal 매칭을 실행한 결과 2000명의 랜덤 회원 시간표가 업로드되었음에도 매칭된 회원 수는 10명 안쪽으로 상당이 낮은 모습을 보인다. meal 목적으로 매칭되기 위해서는 적어도 2교시 3교시가 모두 같거나 3교시 4교시가 모두 같아야 하기 때문이다. meal 목적의 매칭이 활발하게 이루어지게 하기 위해서는 매칭 알고리즘의 난도를 낮출 필요가 있다. 이는 점심시간에 비어있는 시간표가 겹치는 회원과 매칭해 줌으로써 해결할 수 있다.

- study 매칭

: 많은 랜덤유저를 생성했음에도 불구하고 study 매칭에서는 매칭된 인원수가 현저히 적었다. 이는 실제 학생들의 수업이 대체로 6교시 이전에 끝나는 데 반해 랜덤 회원의 6교시 7교시가 무작위로 생성되기 때문이다. 즉 실제 사용자 6교시 이전에 끝난다면 이 사용자와 매칭이 되기 위해서는 적어도 6교시 7교시가 모두 비어있어야 한다. 이러한 경우, 공강 시간을 활용하여 비어 있는 시간이 있어도 매칭되는 조건을 추가하여 매칭 난이도를 낮출 수 있을 것이다.

- friend 매칭

: friend 매칭을 시행하였을 때 실제 회원 기준으로 매칭되는 회원 수는 100명 안팎이었다. 이는 실제 한 요일에 업로드되는 데이터양이 500개이고 매칭 회원 수와 데이터양이 서로 선형 관계에 있음을 가정했을 때 매칭되는 회원 수는 20명 정도로 예상되기에 적절하다고 볼 수 있다. 그러나 수업이 정해져 있는 대학교 특성상 이러한 매칭 방법은 매칭을 여러 번 시도했을 때 이미 매칭된 회원이 여러 번 매칭되는 문제가 있다. 이러한 문제는 여러 번 매칭된 회원을 자동으로 차단하는 기능을 생각해볼 수 있다.

3. 결론

3.1 결론 및 활용 방안

본 프로젝트에서는 GNSS 데이터의 오차를 고려하여 시간표를 설정하였다. 각 건물의 경곗값을 통해 편차를 계산하여 건물의 위치를 설정하였다. 매칭하는 과정은 사용자와 상대의 선호를 모두 고려하여 상호 만족스러운 매칭이 이루어지게 하였으며 타겟을 설정하여 검증하고 수많은 랜덤 회원 정보를 통해 무결성을 확인하였다.

'항소개팅' 프로그램은 학교에서 친구를 만들 수 있는 프로그램이다. 특히 코로나19 등의 사태 이후 복학하여 친구를 사귀고 싶은 많은 학생들이 직접 대면한 상태에서 말을 거는 등의 부담을 덜어줄 수 있다. 만나고 싶은 상대의 조건을 설정하여 같은 과, 다른 과 등 다양한 친구들을 기호에 맞춰 만나볼 수 있다. 본 소개팅 프로그램이 원활히 작동되기 위해선 많은 사용자들이 회원가입을 하고, 자신의 일과를 tracking한 데이터 파일이 업로드 되는 것이 선행되어야 한다. 현재로서는 여러 사용자가 동시 접속하여 실시간으로 업데이트 되어 사람들을 만나볼 수 없지만, 추후 이러한 부분이 서버 개설 등으로 개선된다면 많은 회원들이 이용하기에 간편한 프로그램이 될 것이다.

3.2 향후 과제

'항소개팅' 프로그램은 발전 가능성이 무궁무진하다. 전반적인 프로그램의 틀을 짠 후 나아갈 수 있는 방향과 개선 방안들이 있다.

앞서 언급했다시피 '항소개팅' 프로그램의 가장 큰 단점은 동시에 여러 사용자가 사용하지 못한다는 점에 있다. 로그인 한 회원의 학번이 user_data.txt 파일에 저장되는데, 다른 사용자가 동시에 접속한다면 가장 최근에 로그인 한 회원의 학번이 해당 파일에 저장되며 이후 매칭이 해당 회원 기준으로 시작된다. 하나의 서버에 전체 회원이 로그인 할 때마다 해당 파일에 업데이트하며 저장되기 때문에 여러 사용자의 동시 접속 지원이 불가능하다. 여러 사용자의 동시 접속 지원이 가능하도록 개선할 수 있다.

현재 버전은 선호 옵션들을 선택할 때, 전체 혹은 각 경우에 따라 선택이 가능하다. '다중 선택'이 불가능하게 프로그래밍이 되어 있다. 특정 학과, 특정 학년을 선호하는 사용자의 선호를 충족시키게끔, 확실한 선호 설정을 할 수 있도록 발전시킬 수 있을 것이다. 또한 뒤로 가기 버튼이 없어 모든 상황에서 선택을 필수적으로 해야 하는 단점이 있다.

추가 옵션으로는 이미 매칭된 회원을 그만 매칭하는 차단 기능, 여러 화면에서 앞, 뒤로 가기 버튼, 종료 버튼을 GUI 창에 넣고 그 외의 여러 정보들을 함께 나타낼 수 있다면 기능성이 향상될 것이다.

또한 현재 프로그램에서는 재학생 기준으로 시간표를 설정하였기에 재학생 대상으로만 시간표가 올바르게 작동된다. 대학원생이 서로 매칭될 수 있도록 연구실 매칭 조건을 추가해 볼 수 있고, 동선이 겹치는 경우 매칭되는 알고리즘을 구현하여 보다 많은 회원들에게 매칭될 수 있도록 할 수 있을 것이다. 다른 학교의 건물 정보가 포함되어 있다

면 세종대학교 뿐만 아니라 여러 학교들에서도 교내에서 매칭시켜 주는 프로그램으로 프로그램의 이용 대상이 늘어나며 확장될 수 있다.

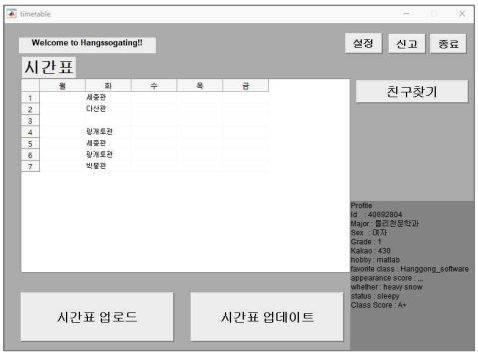


그림 129 업데이트 예시 1

그림 130 업데이트 예시 2

그림 129, 130은 언급한 개선사항이 반영된 GUI 예시이다.

4. 기타

4.1 조원 기여도

- 김찬혁 (21010514) 100%
- 김채은 (21010512) 100%
- 유승표 (21010480) 100%
- 이용신 (21011324) 100%
- 홍재우 (22013571) 100%

전원 동의하에 각 인원이 맡은 바를 1인분 이상 했다고 판단하여 기여도를 평가했다.

4.2 맺음말

- MATLAB으로 실제로 이런 일들을 할 수 있음을 느꼈고, 코드들과 GUI에 대해 많이 배웠다. MATLAB을 이용해서 GPS 데이터를 받고 처리하는 과정도 신기했고, 그 과정을 직접 코드를 작성해서 할 수 있다는 것이 놀라웠다. GUI로 여러 인터페이스를 만들고 하면서 많이 공부했다. 다같이 과제를 하면서 배워가는게 정말 많았던 프로젝트였다.
- MATLAB으로 GUI를 다뤄본 것이 처음이었고, 특히 GNSS 정보를 다뤄본 적이 없어 많은 어려움이 있었다. 코드를 짜면서 끝이 보이지 않는 디버깅이 특히 어려웠다. 결과적으로 하나의 완성된 프로그램을 잘 만든 것 같아 뿌듯하고, 제로 베이스에서 시작해서 팀원들과 함께 완성한 이 경험을 바탕으로 이제 어떤 코딩이든 잘할 수 있을 것 같은 자신감이 생겼다.

- 미리미리 준비한다고 조금씩 한 거 같은데, 생각보다 오래 걸렸다고 생각한다. 참 힘들었고, 다들 열심히 끝까지 잘해주셔서 정말 감사하다. 또한 이번 경험을 통해 코딩에 있어 자신감이 생겼다. 이후 MATLAB을 사용할 때 이번 경험을 바탕으로 잘 할 수 있을 것 같다.
- GNSS 위성 항법 시스템에 대해 자세히 알아볼 수 있어 좋았다. 그중에 최근 스마트폰은 GNSS 통합시스템을 사용하여 20개가 넘는 많은 위성의 신호를 받아올 수 있다는 것을 알았고 이를 A-GPS 기술을 사용해 더욱 빠르게 통신하는 시스템을 알게 되었다. 직접 위성 신호 오차를 분석하고 이를 보완하는 과정을 거치며 위성 신호에 오차가 생기는 이유를 알게 되었고, 이를 보완하는 방법을 알게 되었다. 나아가 최근에는 기지국과의 위치 데이터 통신, 칼만 필터 보정을 통해 비행기나 기차 등 빠르게 움직이는 상황에서도 위치를 정확하게 추적할 수 있다는 것을 알게 되었다. 매트랩 매칭 시스템을 구성하면서 매칭 옵션과 같이 여러 경우의 수에서 이를 분류하는 방법을 공부하는 계기가 되었다. 또 컴퓨터 프로그래밍 시스템에서 하드웨어 저장소의 파일 위치를 다루는 법을 배울 수 있었다.
- GUI 파트를 담당하며 기존의 딱딱했던 부분도 폰트 수정과 다양한 요소들을 활용하면 보다 직관적이거나 친근감 있게 사용자에게 다가올 수 있다는 사실을 알았다. 기존에 구상하려던 것들을 실제로 만들어 보며 오류 수정과 어려움이 많았지만, 팀원들과 함께 수정해 가며 부족한 부분을 채울 수 있었다.

5. 프로그램 코드

아래의 프로그램 코드들은 별도의 파일로 첨부하였다. 로컬에서 작동이 가능하게 설정되어 있다. (CSV 데이터 파일을 'upload' 폴더에 저장해야 함.) 'database' 파일에는 랜덤 검증 시 사용했던 데이터베이스와 타겟팅 검증 시 사용한 데이터베이스 파일이 함께 포함되어 있다. 개선된 GUI 코드는 '항소개팅_v2' 폴더에 포함하였다. (해당 폴더 내에서 코드가 전부 작동되게 셋팅되어 있음.)

- example.m / register.m / login.m / modify.m / time_table.m / mainhome.m / setting.m / matching_gui.m / result.m
- example.fig / register.fig / login.fig / modify.fig / time_table.fig / mainhome.fig / setting.fig / matching_gui.fig / result.fig
- converter.m / creatcourse.m / coursetotimetable.m / timetabletotext.m / randomuser.m

6. 출처

[1] 국토지리정보원, 2016, 글로벌 환경변화 대응을 위한 GNSS 인프라 진단 연구 보고서

[2] Stuido ODOC, (2024, April 14) *[Useful] point in polygon algorithm (javascript)* https://studiodoc.tistory.com/167